A

Major Project

On

DEEP ACOUSTIC ANOMALY DETECTION IN INDUSTRIAL MACHINERY USING HUBERT-DRIVEN HYBRID TREE LEARNING

(Submitted in partial fulfilment of the requirements for the award of Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

L. Bavitha (227R1A12A3)

K. Mahesh (227R1A12A0)

V. Arjun Das (237R5A1209)

Under the Guidance of

Mrs. K. Supriya Suhasini

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)

Recognized Under Section 2(f) & 12(B) of the UGCAct.1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

May, 2026.

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled "**DEEP ACOUSTIC ANOMALY DETECTION IN INDUSTRIAL MACHINERY USING HUBERT-DRIVEN HYBRID TREE LEARNING**" being submitted by **L. Bavitha (227R1A12A3), K. Mahesh (227R1A12A0) & V. Arjun Das (237R5A1209)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mrs. K. Supriya Suhasini
Assistant Professor
INTERNAL GUIDE **HEAD OF THE DEPARTMENT**

Dr. A. Raji Reddy
DIRECTOR **Signature of External Examiner**

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mrs. K. Supriya Suhasini,** Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Dr. J. Narasimharao, Mrs. K. Supriya Suhasini, Mr. D. Krishna Kumar, Mr. G. Shankar, Mrs. M. Siva Jyothi and Ms. G. Menaka** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our **Head of the Department**, for encouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

L. Bavitha (227R1A12A3)
K. Mahesh (227R1A12A0)
V. Arjun Das (237R5A1209)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.

2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.

3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

DEPARTMENT MISSION:

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problems.

2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.

3. To enhance the leadership qualities and team spirit with social values to serve the society.

ABSTRACT

Machine condition monitoring using acoustic signals has become increasingly important for detecting early mechanical faults in industrial machinery. Industries require reliable and automated systems to identify potential failures before they cause major damage. Traditional monitoring methods such as manual inspection, vibration sensors, and rule-based analysis often fail to capture subtle acoustic variations that occur before equipment failure. Industrial machines like engines, compressors, and motors produce complex sound patterns during operation, making fault detection more challenging. Conventional MFCC-based machine learning models such as Logistic Regression and Linear Discriminant Analysis are commonly used for analyzing machine sounds. However, these models rely heavily on handcrafted features and struggle to differentiate between acoustically similar faults such as air leaks, idling disturbances, and oil leaks. Their performance is also affected by background noise and limited generalization ability, which leads to misclassification and lower accuracy. To overcome these limitations, this project proposes a hybrid framework that combines HuBERT-based deep audio embeddings with a TAO Tree classifier. HuBERT extracts rich contextual audio features from machine sound signals, while the TAO Tree provides strong non-linear classification capability. The system is implemented with a Tkinter graphical interface that allows dataset upload, feature extraction, model training, evaluation, and prediction. Experimental results show that the hybrid HuBERT and TAO Tree model achieves very high performance with 100% accuracy across all fault categories. This approach effectively captures complex acoustic patterns and improves the reliability of machine fault detection. Overall, the proposed system provides an accurate and efficient solution for intelligent machine condition monitoring and predictive maintenance in modern Industry 4.0 environments.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of Deep Acoustic Anomaly Detection in Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning.	13
Figure 3.3.1	Class Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning	15
Figure 3.3.2	Use Case Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning	16
Figure 3.3.3	Sequence Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning	17
Figure 3.3.4	Activity Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning	18

LIST OF SCREENSHOTS

SCREENSHOT NO	SCREENSHOT NAME	PAGE N0
Screenshot 4.1	Screenshot of the Dataset directory structure with folders 'air leak, background noise, idiling, normal, oil leak ' having all the training examples	21
Screenshot 4.2	Screenshot of the "Air leak" Folder Showing Sample audio Files	21
Screenshot 4.3	Screenshot of the "Air leak" Folder Showing Sample audio Files	21
Screenshot 4.4	Screenshot of the "idling" Folder Showing Sample audio Files	22
Screenshot 4.5	Screenshot of the "normal " Folder Showing Sample audio Files	22
Screenshot 4.6	Screenshot of the "oil leak" Folder Showing Sample audio Files	22
Screenshot 5.1	Screenshot of the Main GUI Interface of the Acoustic Fault Detection System.	35
Screenshot 5.2	Screenshot of the Dataset Upload Confirmation and Category Detection Display.	36
Screenshot 5.3	Screenshot of the Loaded MFCC Feature Arrays (X and Y) with Class Distribution Plot.	37
Screenshot 5.4	Screenshot of the Train–Test Split Distribution for Acoustic Fault Dataset.	37
Screenshot 5.5	Screenshot of the Logistic Regression – Confusion Matrix for Acoustic Fault Classification.	38

Screenshot 5.6	Screenshot of the Logistic Regression – ROC Curve (Per-Class Performance).	38
Screenshot 5.7	Screenshot of the Linear Discriminant Analysis (LDA) – Confusion Matrix.	39
Screenshot 5.8	Screenshot of the Linear Discriminant Analysis (LDA) – ROC Curve (Per-Class Performance).	40
Screenshot 5.9	Screenshot of the HUBERT + Tao Tree – Confusion Matrix.	40
Screenshot 5.10	Screenshot of the HUBERT + Tao Tree roc curves.	41
Screenshot 5.11	Screenshot of the Predicted Output Waveform for Audio Sample: Air Leak.	41
Screenshot 5.12	Screenshot of the Predicted Output Waveform for Audio Sample: Normal Engine Sound.	42

LIST OF TABLES

TABLE N0	TABLE NAME	PAGE NO
Table 6.2.1	Upload Dataset	56
Table 6.2.2	Feature Extraction	56
Table 6.2.3	Data Splitting	56
Table 6.2.4	Model Training	57
Table 6.2.5	Anomaly Prediction	57

TABLE OF CONTENTS

	PAGE N0
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF SCREENSHOTS	iii
LIST OF TABLES	v
1. INTRODUCTION	1
1.1 PROJECT PURPOSE	1
1.2 PROJECT FEATURES	2
2. LITERATURE SURVEY	3
2.1 REVIEW OF RELATED WORK	7
2.2 DEFINITION OF PROBLEM STATEMENT	8
2.3 EXISTING SYSTEM	9
2.4 PROPOSED SYSTEM	10
2.5 OBJECTIVES	11
2.6 HARDWARE & SOFTWARE REQUIREMENTS	12
2.6.1 HARDWARE REQUIREMENTS	12
2.6.2 SOFTWARE REQUIREMENTS	12
3. SYSTEM ARCHITECTURE & DESIGN	13
3.1 PROJECT ARCHITECTURE	13
3.2 DESCRIPTION	14
3.3 UML DIAGRAMS	15
3.3.1 CLASS DIAGRAM	15
3.3.2 USE CASE DIAGRAM	16
3.3.3 SEQUENCE DIAGRAM	17
3.3.4 ACTIVITY DIAGRAM	18
4. IMPLEMENTATION	19
4.1 ALGORITHMS USED	19
4.2 SAMPLE CODE	23
5. RESULTS & DISCUSSION	35
6. VALIDATION	43
6.1 INTRODUCTION	43
6.2 TEST CASES	44
6.2.1 UPLOADING DATASET	44
6.2.2 CLASSIFICATION	44
6.2.3 DATA SPLITTING	44
6.2.4 MODEL TRAINING	45
6.2.5 ANOMALY PREDICTION	45
7. CONCLUSION & FUTURE ASPECTS	46
7.1 PROJECT CONCLUSION	46
7.2 FUTURE ASPECTS	47
8. BIBLIOGRAPHY	48
8.1 REFERENCES	48
8.2 GITHUB LINK	50

1. INTRODUCTION

1. INTRODUCTION

Early detection of mechanical faults in industrial environments is essential to prevent catastrophic failures, reduce maintenance costs, and improve operational efficiency. Traditional monitoring methods such as manual inspection and threshold-based vibration analysis often fail to identify subtle or early-stage defects. With advances in digital sensing and artificial intelligence, acoustic-based machine fault detection has become an effective and non-intrusive approach for monitoring equipment such as compressors, pumps, and engines. Anomalous Sound Detection (ASD) identifies whether machine sounds are normal or abnormal and is widely used in applications like audio surveillance and machine condition monitoring. Since machines typically operate under normal conditions and anomalous sound data is difficult to collect, ASD is often treated as an unsupervised learning task, where models are trained only on normal sound data. The DCASE 2020 challenge focused on unsupervised detection of anomalous machine sounds using datasets such as ToyADMOS and MIMII, which contain 10-second single-channel recordings of machine operation mixed with environmental noise. The goal of this work is to develop an unsupervised classification model capable of accurately detecting abnormal machine sounds and outperforming previously proposed systems.

1.1 PROJECT PURPOSE

The purpose of this project, "Deep Acoustic Anomaly Detection in Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning," is to develop an intelligent system that can automatically detect faults in industrial machines by analyzing the sounds they produce during operation. Industrial machines often generate specific acoustic patterns when they function normally, and any abnormal sound may indicate a potential fault or malfunction. Traditional monitoring methods rely on manual inspection or basic sensor measurements, which are not always effective for early fault detection.

This project aims to use advanced machine learning techniques, particularly HuBERT-based deep audio feature extraction combined with a TAO Tree classification model, to identify abnormal sound patterns accurately. By processing and analyzing machine audio signals, the system can detect anomalies in real time and help industries perform predictive maintenance, reduce equipment failure, minimize downtime, and improve operational efficiency. The project also provides a user-friendly interface for uploading data, training models, and performing predictions.

1.2 PROJECT FEATURES

The project includes several advanced features that help in detecting faults in industrial machinery through acoustic analysis. It is designed to collect and process machine sound recordings and perform audio preprocessing steps such as noise reduction, normalization, and signal cleaning to improve the quality of the audio data. The system extracts important audio features using traditional techniques like Mel-Frequency Cepstral Coefficients (MFCC) along with advanced deep learning-based HuBERT embeddings, which capture complex acoustic patterns from machine sounds.

Another important feature of the project is the use of multiple machine learning algorithms for classification. Models such as Logistic Regression, Linear Discriminant Analysis (LDA), and a TAO Tree (Random Forest) classifier are used to train the system and identify whether a machine is operating normally or showing signs of anomalies. These models analyze the extracted features and classify machine conditions with high accuracy, helping industries detect potential faults at an early stage.

The system also includes a user-friendly graphical interface developed using Tkinter, allowing users to easily upload datasets, perform feature extraction, train machine learning models, and test new audio samples for anomaly detection. In addition, the project provides performance evaluation through metrics like accuracy, confusion matrix, and graphical visualization, making the system practical and effective for real-time industrial machine monitoring.

2.LITERATURE SURVEY

2. LITERATURE SURVEY

Previous studies on machine fault detection have widely used acoustic signal analysis and machine learning techniques, including deep learning models, autoencoders, CNNs, and attention-based frameworks, to detect abnormal machine sounds. These methods analyze features such as spectrograms, MFCCs, and audio embeddings and have achieved high performance on datasets like MIMII and ToyADMOS. Signal processing techniques such as FFT and GLRT have also been used to capture the physical characteristics of acoustic signals for reliable fault detection.

Acoustic monitoring offers advantages over traditional sensor-based methods because microphones are low-cost, non-invasive, and easy to deploy. While it has been successfully applied in areas such as CNC machining, turbine engines, and industrial automation, challenges like background noise, varying operating conditions, and limited noise-robust solutions still exist, highlighting the need for more advanced data-driven approaches for accurate real-world anomaly detection.

Wang et al.[1] proposed two classification-based anomaly detection models: an outlier classifier to distinguish abnormal sounds from normal ones and an ID classifier that detects anomalies using classification confidence and hidden embedding similarity. Their ensemble approach achieved 95.82% AUC and 92.32% pAUC in the DCASE 2020 Task 2 dataset, outperforming existing state-of-the-art models.

Jombo, et al.[2] discussed the development of acoustic-based fault diagnostic techniques for machine condition monitoring. The study highlights that traditional monitoring methods rely on contact sensors such as accelerometers and pressure or temperature sensors, which may be difficult to install due to cost, space, or reliability issues. Acoustic monitoring using microphones offers a non-contact, low-cost alternative, but challenges such as background noise and varying operating conditions must be addressed for effective industrial application.

Pichler et al.[3] proposed a fault detection method based on the physical characteristics of acoustic signals, modeling fault-related sounds as exponentially decaying oscillations. Using this model, they developed a detection approach based on the Generalized Likelihood Ratio. Test (GLRT). Experiments with synthetic and real-world steel industry data showed that the GLRT-based method outperformed traditional audio features, particularly in high-noise environments, achieving significantly higher detection performance with lower false-positive rates.

Ahmed et al.[4] proposed an acoustic-based condition monitoring method for an APU ignition system using experimental data from a Boeing 737-400 aircraft. The approach includes noise suppression, spark frequency estimation, event segmentation, and feature extraction to analyze ignition behavior. The results showed that the method can detect inconsistencies in the ignition exciter and identify degradation trends in spark acoustic signals.

Piankitrungreang et al.[5] developed an acoustic-based monitoring system for high-speed CNC drilling to detect machine states in real time. High-quality acoustic sensors capture drilling sounds, enabling identification of events like tool engagement and material breakthrough. Signal processing techniques such as spectrogram analysis and FFT extract dominant sound patterns. DBSCAN clustering is used to classify operational states like cutting and returning. Experiments on different materials showed distinct acoustic profiles. The study demonstrated that acoustic signals can support non-invasive monitoring and predictive maintenance.

Grigoriev et al.[6] studied acoustic signals generated during different material processing operations. Their analysis showed that changes in machining conditions affect the spectral composition of acoustic signals. Factors such as tool wear, erosion products in WEDM, and laser focal plane shifts influence the ratio of low- and high-frequency amplitudes. These acoustic variations reflect the quality of the processing operation. The findings indicate that monitoring spectral characteristics can help detect processing abnormalities. This approach can be applied in automated monitoring systems for industrial equipment.

Zhou et al.[7] proposed an unsupervised anomaly detection method using audio latent representations and a parallel attention mechanism. An autoencoder extracts low-dimensional features from machine sound signals. A phase-aware attention block dynamically weights these features to improve anomaly detection sensitivity. The framework also uses adversarial training and dual encoding to enhance robustness. Experiments on datasets such as MIMII, ToyADMOS, and wind turbine data achieved high AUC scores.

Liu et al.[8] introduced a lightweight deep learning model called the Attention-Based Deep Convolutional Autoencoding Prediction Network (AT-DCAEP). The model combines convolutional autoencoders for feature extraction and attention mechanisms for prediction. It is designed for anomaly detection in multivariate time series data without requiring large labeled datasets. The architecture efficiently captures complex data patterns. Experiments on six public datasets demonstrated superior performance compared to existing approaches.

Choi et al.[9] proposed an unsupervised machine learning approach for detecting anomalous behavior in Industrial Control Systems (ICS). The method uses a composite autoencoder model to learn normal system behavior patterns. It can detect anomalies without relying on labeled training data. Experiments were conducted on HIL-augmented ICS datasets to evaluate performance. The model successfully identified abnormal patterns related to both value and time features. This approach improves system reliability and supports efficient monitoring of industrial processes.

Thoidis et al.[10] proposed a feature learning approach for machinery sound analysis using deep temporal convolutional neural networks. The model learns descriptive audio embeddings directly from raw sound signals. A supervised feature learning method is combined with an unsupervised one-class neural network for anomaly detection. The approach also utilizes spatial audio information from circular microphone arrays. This improves the ability to detect anomalies across different machine states. Experiments on the MIMII dataset achieved a mean AUC score of 91%.

Tagawa et al.[11] presented a noise-tolerant anomaly detection approach for industrial machine sound data. The method uses a Generative Adversarial Network (GAN) to reconstruct sound signals and detect abnormalities. Additional techniques such as the Unscented Kalman Filter and MSE loss with L2 regularization improve model performance. Experiments were conducted using the MIMII dataset. The proposed approach outperformed baseline methods like OC-SVM and Autoencoder–Decoder models. Results showed improved AUC values for noisy pump data.

Van Truong et al.[12] proposed an improved anomaly detection method for machine sound analysis. Traditional autoencoder approaches using log-Mel spectrograms struggle with non-stationary machine sounds. To address this, the authors introduced a Mixed Feature representation combining multiple audio features. They also developed a Fully Connected U-Net autoencoder architecture. Experiments showed improved AUC and pAUC performance compared to baseline models. The optimized model significantly enhanced anomaly detection across different machines.

Lee et al.[13] proposed a noise-robust fault diagnosis method using Variational Autoencoder (VAE) and Domain Adversarial Neural Network (DANN). The system analyzes noisy acoustic signals from industrial machines. MFCC-based statistical features were extracted to represent machine sound characteristics. Noise reduction techniques such as magnitude spectral subtraction were also applied. The model learns normal machine behavior and detects faults through reconstruction errors.

Bernardes et al.[14] studied the use of acoustic emission sensors to analyze tool damage during milling operations. Acoustic signals were processed using techniques such as FFT, RMS calculations, and Butterworth filters. These signal processing methods helped design condition indicators for tool condition monitoring. The approach aims to detect tool wear and damage during manufacturing processes. Results indicated potential improvements in process optimization and machine efficiency. However, the study presents partial results as the implementation is still ongoing.

Ahmed et al.[15] used spectrograms of airborne acoustic emission signals and a CNN-based Residual Network to monitor tool health in end-milling machines. The dataset included signals from new, moderately used, and worn-out cutting tools. Experiments were conducted using hardwood and softwood materials under different cutting depths. Acoustic signals were recorded and analyzed for machine condition classification. The trained model achieved very high prediction accuracy of 99.7%. The results demonstrate the effectiveness of deep learning for tool health monitoring using acoustic data.

Qurthobi et al.[16] conducted a systematic review of acoustic-based approaches for mechanical failure detection in industrial machines. The study analyzed 52 research articles collected from IEEE Xplore, ScienceDirect, and SpringerLink using the PRISMA review methodology. The review examined recent developments and structural extensions in acoustic monitoring techniques.

Ciaburro et al.[17] reviewed methodologies related to acoustic emission techniques for monitoring materials and structural conditions. The study analyzed recent developments in machine learning approaches used for damage detection and localization. These techniques help characterize fracture behavior and predict failure modes in structures. The research highlighted the strong adoption of acoustic emission methods in structural health monitoring. The findings confirmed that AE-based techniques are effective for identifying damage even in noisy environments. This demonstrates their importance in modern monitoring systems.

Alharbi et al.[18] presented a review of acoustic and vibration signal-based fault detection methods for belt conveyor idlers using machine learning models. The study discussed key steps involved in these approaches, including data collection, signal processing, feature extraction, and model development. It also described the structure and common failure sources of conveyor idlers. The research emphasized the increasing use of machine learning for detecting idler defects in industrial systems. Acoustic and vibration signals are commonly used for condition monitoring.

Dado et al.[19] investigated the use of acoustic emission signals for real-time monitoring and control of CNC milling processes. Experiments were conducted using MDF material on a five-axis CNC machine while capturing AE signals with dual-channel sensors. The study analyzed how spindle speed and feed rate affect the characteristics of acoustic signals. Results showed that higher machining parameters produced increased signal amplitudes and distinct frequency peaks. Statistical analysis confirmed a relationship between AE signal magnitude and cutting conditions. The findings suggest that AE monitoring can improve machining efficiency, tool life, and predictive maintenance.

Umar et al.[20] proposed a fault diagnosis method for milling machines using acoustic emission signals and a hybrid deep learning model. AE signals are converted into CWT scalograms and processed using Gaussian filtering for noise reduction. A VGG16-based CNN extracts spatial features, while a BiLSTM network captures temporal patterns. A genetic algorithm is used to optimize feature selection and improve model performance. The approach enables accurate fault detection and supports predictive maintenance in milling machines.

2.1 REVIEW OF RELATED WORK

1. Use of Acoustic Signals for Machine Monitoring

Many researchers have used acoustic signals to monitor the condition of industrial machines. Machines produce different sound patterns during normal and faulty operations, and these sound variations can indicate potential problems. By analyzing these acoustic signals, researchers can identify abnormal machine behavior without physically interfering with the machine. This method is considered a non-invasive and cost-effective approach for machine health monitoring.

2. Application of Signal Processing Techniques

Several studies have applied signal processing methods such as Fast Fourier Transform (FFT), Root Mean Square (RMS), and spectral analysis to analyze machine sounds. These techniques help convert raw audio signals into meaningful information by identifying frequency patterns and signal characteristics. This makes it easier to detect unusual sound patterns that may indicate machine faults.

3. Use of Machine Learning Algorithms

Machine learning algorithms have been widely used to classify machine conditions based on extracted audio features. Algorithms like Logistic Regression, Support Vector Machines, and Random Forest have been used in earlier research to detect whether a machine is operating normally or showing signs of failure. These models improve detection accuracy by learning patterns from historical data.

4. Adoption of Deep Learning Models

Recent research has focused on deep learning techniques such as Convolutional Neural Networks (CNNs) and autoencoders for anomaly detection. These models can automatically learn complex patterns from large audio datasets and identify subtle differences between normal and abnormal sounds. Deep learning approaches often achieve better performance than traditional machine learning models.

5. Challenges in Existing Methods

Although many techniques have been developed for acoustic fault detection, existing systems still face challenges such as sensitivity to background noise, limited feature extraction capability, and difficulty distinguishing between similar fault types. These limitations highlight the need for more advanced methods, such as combining deep learning-based feature extraction with robust classification models to improve detection accuracy and reliability.

2.2 DEFINATION OF PROBLEM STATEMENT

A problem statement is a clear and concise description of an issue or challenge that needs to be addressed through a project or research. It explains the existing problem, the limitations of current methods, and why the problem needs to be solved. The problem statement helps to define the main focus of the project and guides the development of appropriate solutions to overcome the identified issues. In simple terms, it describes what the problem is, why it occurs, and the need for a better solution.

2.3 EXISTING SYSTEM

Traditional machine-fault detection systems rely mainly on manual inspection, basic sensor readings, and fixed threshold-based monitoring. Industries depended heavily on human operators to listen, observe, and physically examine machines to identify abnormalities. Simple devices like vibration meters, thermometers, and RPM counters were used to assess machine performance, but these tools could not detect deeper internal faults or emerging anomalies. This approach works only when faults are obvious and manually detectable.

Traditional systems operate in a sequential manner. First, operators perform manual listening to detect abnormal sounds like rattling, knocking, leaking, or continuous hums. Next, periodic inspections are conducted where technicians visually check the machine, measure vibration and temperature, and verify mechanical condition. The system then uses predefined threshold values for vibration, heat, or speed to trigger alerts when measurements exceed safe limits.

Once an abnormal condition is observed or a threshold alert is raised, experts manually diagnose the cause of the fault. This process relies entirely on skill, experience, and subjective interpretation. The final step involves scheduling maintenance, repairing parts, or stopping the machine. Because everything depends on manual judgment and non-adaptive thresholds, this traditional system suffers from delays, missed faults, and inconsistent results.

LIMITATIONS OF EXISTING SYSTEM

- Completely dependent on human skill, judgement, and experience, leading to inconsistent results.
- High risk of missing early-stage faults that produce subtle changes not easily detected manually.
- Fixed thresholds do not adapt to machine age, load variations, or environmental changes.
- Ineffective in noisy industrial environments where human listening becomes unreliable.
- Manual inspection is time-consuming and cannot support continuous 24/7 monitoring.
- Not scalable for large industries with hundreds of machines requiring regular monitoring.
- High chances of unexpected breakdowns due to late detection.

- No ability to learn patterns or improve over time because it lacks data-driven analysis.
- Frequent false alarms or missed alarms due to rigid rules and non-adaptive measurements.
- No detailed insights into internal machine behavior because only surface-level parameters are measured.

2.4 PROPOSED SYETEM

The proposed system is designed to detect machine faults using acoustic signal analysis and machine learning techniques. Initially, machine audio data is collected from equipment operating under different conditions such as normal operation, idle state, air leakage, oil leakage, and background mechanical noise. Collecting high-quality audio is important because collection, preprocessing steps such as noise removal, normalization, and segmentation are machine sounds contain valuable information about internal mechanical behavior. After data performed to clean the recordings and ensure that the audio signals are consistent and suitable for analysis.

Following preprocessing, the system performs feature extraction using both traditional signal-processing methods and deep learning techniques. Traditional features such as MFCC, spectral centroid, spectral bandwidth, spectral contrast, chroma, RMS energy, and spectral roll-off are extracted to capture frequency, energy, and tonal characteristics of the sound signals. In addition to these handcrafted features, the system uses HuBERT (Hidden Unit BERT), a transformer-based deep learning model, to generate deep audio embeddings that capture complex acoustic patterns and contextual information. Combining these shallow and deep features provides a richer representation of machine sound signatures.

Once the features are extracted, machine learning models such as Logistic Regression, Linear Discriminant Analysis (LDA), and Random Forest are trained to classify machine conditions. These models learn patterns associated with different machine states and are evaluated using performance metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC curves. After validation, the trained system can perform real-time prediction, where new audio recordings are processed through the pipeline to identify machine conditions instantly. The final output includes predicted machine states, waveform visualization, and detailed fault identification, making the system a reliable and scalable solution for machine condition monitoring and predictive maintenance.

ADVANTAGES OF PROPOSED SYSTEM

- Extracts deep contextual representations of machine audio signals using HUBERT, capturing frequency, temporal, and semantic structure far beyond MFCC capabilities.
- Highly robust to background noise and environmental variations because transformer embeddings encode meaningful acoustic patterns.
- Provides significantly better generalization for unseen machine faults due to the pre-trained transformer's ability to understand diverse audio characteristics.
- Mean-pooled HUBERT embeddings offer a compact yet information-rich representation that improves model discrimination power.
- Random Forest on HUBERT embeddings creates strong decision boundaries and handles class imbalance more effectively than classical ML models.
- Reduces misclassification of acoustically similar faults (e.g., air leak vs. normal) through high-dimensional deep features.
- Achieves faster inference during real-time monitoring because embeddings are computed once and classification is lightweight.
- Highly scalable and extendable, allowing new fault types to be added without redesigning the feature extraction pipeline.

2.5 OBJECTIVES

- **Develop an acoustic anomaly detection system**- The project aims to design a system that can detect abnormal sounds in industrial machines and identify possible faults during machine operation.
- **Extract important audio features from machine sounds**- The system will extract meaningful audio features such as MFCC and HuBERT embeddings to represent machine sound patterns effectively.
- **Apply machine learning algorithms for classification**- Machine learning models will be used to analyze the extracted features and classify whether the machine condition is normal or faulty.

- **Improve accuracy and reliability of fault detection**- The project focuses on improving detection accuracy by combining advanced feature extraction techniques with efficient classification methods.
- **Provide a user-friendly interface**- A graphical interface will be developed so that users
- can easily upload audio data, train the model, and detect machine anomalies.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS

Hardware interfaces specifies the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

- Processor : Intel Core i3
- Hard disk : 512GB.
- RAM : 4GB.

2.6.2 SOFTWARE REQUIREMENTS

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

- Python : 3.12.0
- Operating system : Windows
- GUI : Tkinter

3.SYSTEM ARCHITECTURE & DESIGN

3.SYSTEM ARCHITECTURE & DESIGN

Project architecture refers to the structural framework and design of a project, encompassing its components, interactions, and overall organization. It provides a clear blueprint for development, ensuring efficiency, scalability, and alignment with project goals. Effective architecture guides the project's lifecycle, from planning to execution, enhancing collaboration and reducing complexity.

3.1 PROJECT ARCHITECTURE

This project architecture shows the procedure followed

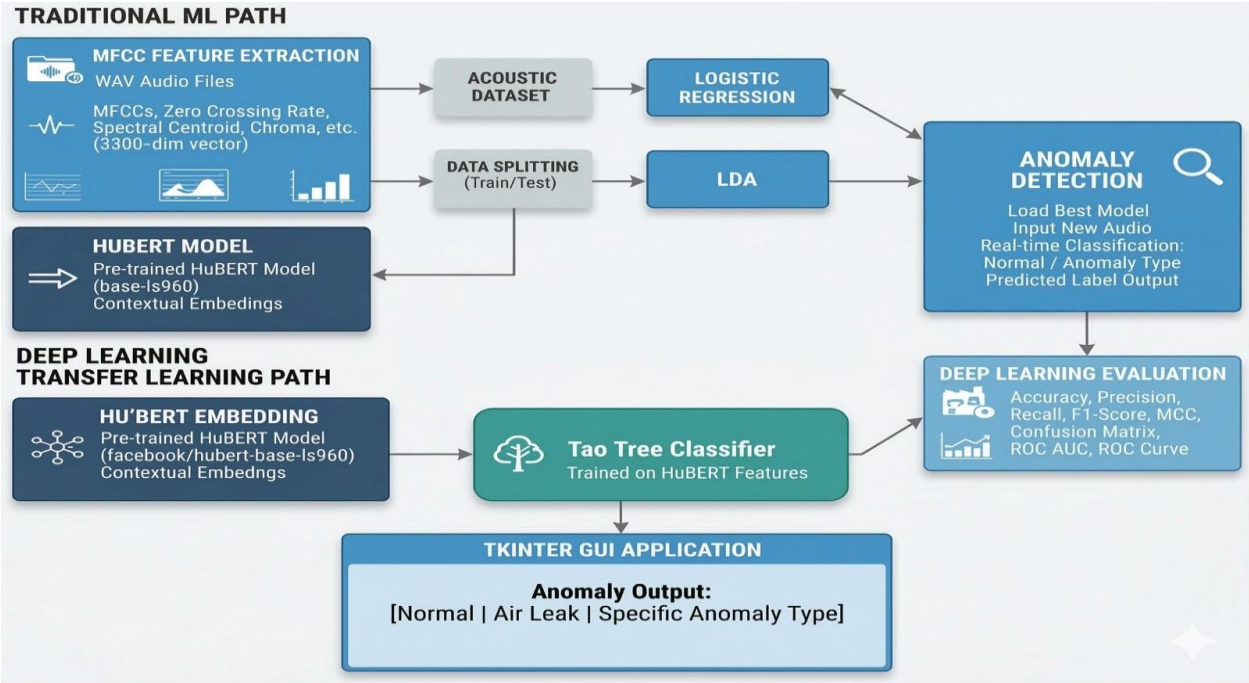


Figure 3.1: Project Architecture of Deep Acoustic Anomaly Detection in Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning.

3.2 DESCRIPTION

- **Audio Data Input**- The system begins with collecting WAV audio files from industrial machines. These audio recordings contain the sound signals generated during machine operation and are used as the input data for the anomaly detection system.
- **Feature Extraction**- The audio files are processed to extract important sound features such as MFCC, Zero Crossing Rate, Spectral Centroid, Chroma, and other acoustic features. These features convert raw audio signals into numerical data that can be analyzed by machine learning models.
- **Dataset Preparation and Splitting**- The extracted features are organized into an acoustic dataset. The dataset is then divided into training and testing sets to train the models and evaluate their performance.
- **Traditional Machine Learning Models**- In the traditional ML path, algorithms such as Logistic Regression and Linear Discriminant Analysis (LDA) are trained using the extracted features. These models learn the patterns of normal and abnormal machine sounds.
- **Deep Learning Feature Extraction**- In the deep learning path, a pre-trained HuBERT model is used to generate contextual audio embeddings from the machine sounds. These embeddings capture deeper acoustic patterns compared to traditional features.
- **Classification using TAO Tree Classifier**- The HuBERT embeddings are used to train a TAO Tree classifier, which performs the final classification of machine conditions as normal or abnormal.
- **Anomaly Detection**- The trained model analyzes new audio samples and identifies whether the sound represents normal machine operation or a specific fault condition.
- **Performance Evaluation**- The system evaluates model performance using metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC curve.
- **GUI Output Display**- Finally, the results are displayed through a Tkinter GUI application, where the predicted machine condition (such as normal, air leak, etc.) is shown to the user.

3.3 UML DIAGRAMS

Unified Modeling Language (UML) is a standardized visual language used to model, design, and document the architecture of software systems. It provides a set of graphical notations to represent the structure and behavior of a system, making complex systems easier to understand and communicate among developers, stakeholders, and business analysts.

3.3.1 CLASS DIAGRAM

A class diagram in UML serves as a blueprint for defining the static structure of a system by illustrating its classes, their attributes, methods, and the relationships among them. Each class represents a key entity in the system, with attributes describing its properties and methods outlining its behaviors or functionalities. Relationships like "is-a" (inheritance) or "has-a" (association) clarify how classes interact, enabling developers to refine the system's design based on use case requirements.

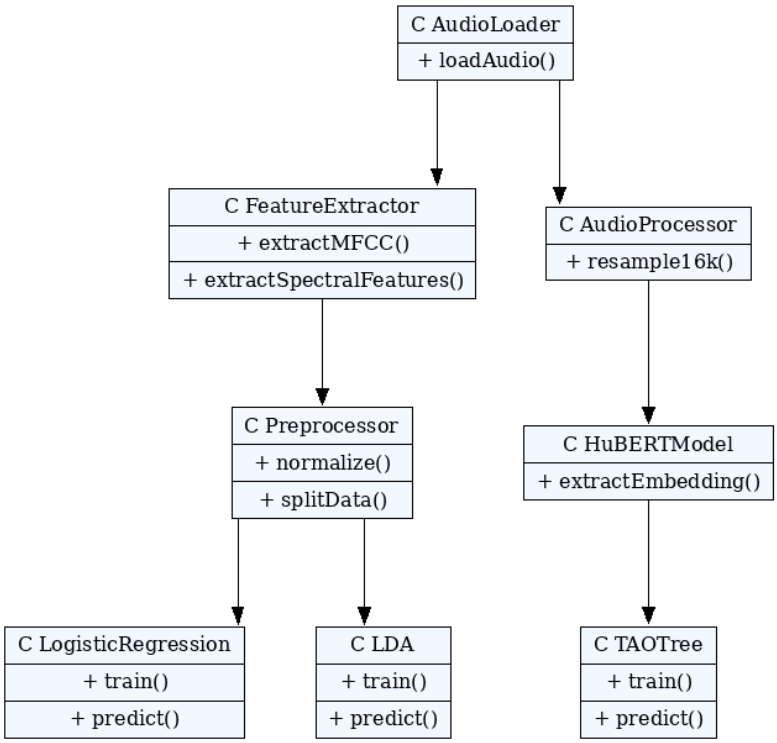


Figure 3.3.1: Class Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning

3.3.2 USE CASE DIAGRAM

A use case diagram in UML offers a high-level, behavioral view of a system by depicting its actors, use cases, and their interactions. Actors, which can be users or external systems, represent entities interacting with the system, while use cases describe specific functionalities or goals the system supports, such as "Login" or "Process Payment." The diagram highlights dependencies, such as "includes" or "extends" relationships, to show how use cases relate, providing a clear picture of what the system does and who it serves. This makes it an essential tool for stakeholders to understand system functionality without diving into technical details.

For AI systems, use case diagrams are instrumental in outlining how users or other systems interact with AI functionalities. For instance, in a chatbot system, actors like "User" or "Admin" might engage with use cases like "Send Message" or "Train Model." The diagram helps developers and stakeholders align on the AI system's scope, ensuring that critical interactions, such as user queries or model updates, are captured early in the design process, facilitating clear communication and requirement validation.

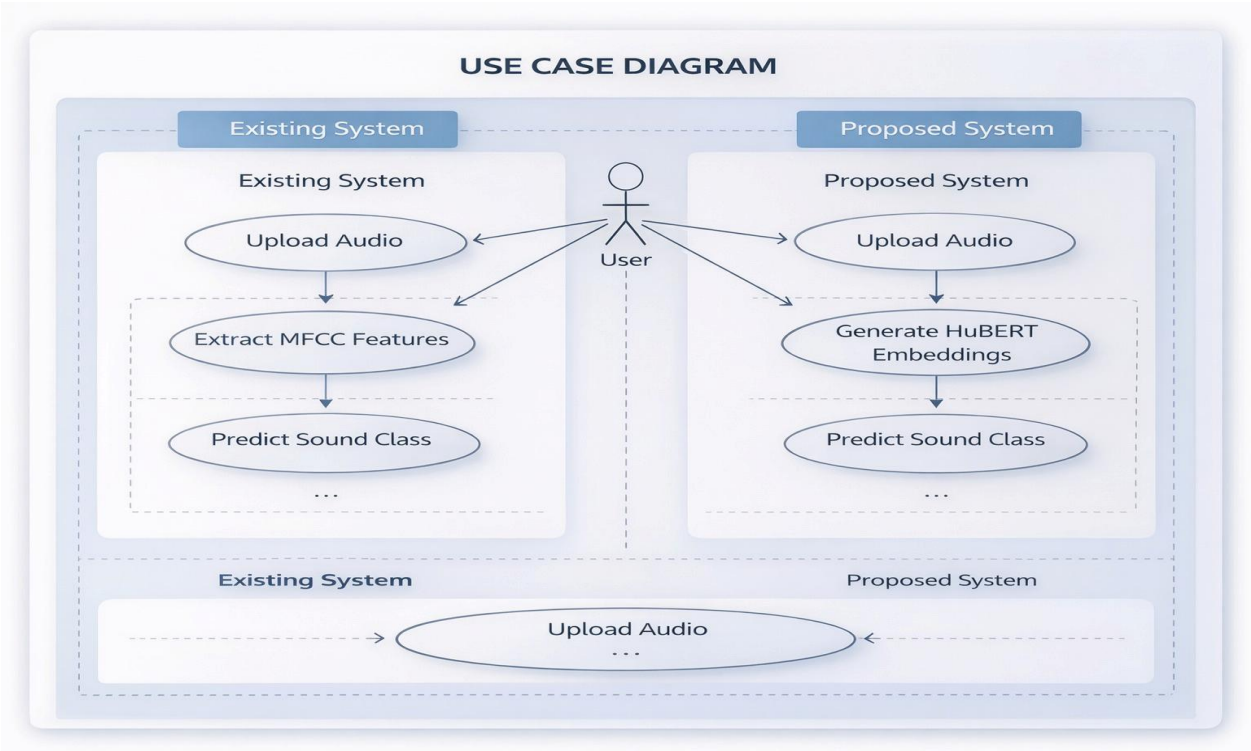


Figure 3.3.2: Use Case Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning

3.3.3 SEQUENCE DIAGRAM

A sequence diagram in UML illustrates the dynamic interactions between objects or processes in a system, focusing on the order of messages exchanged over time. Represented by vertical lifelines for each object and horizontal arrows for messages, it captures the flow of operations, such as method calls or data exchanges, in a specific scenario. This makes it ideal for detailing how components collaborate to achieve a particular function, providing a clear, step-by-step visualization of system behavior. In AI applications, sequence diagrams are particularly useful for modeling the runtime behavior of complex processes, such as how a Outputn system processes user input.

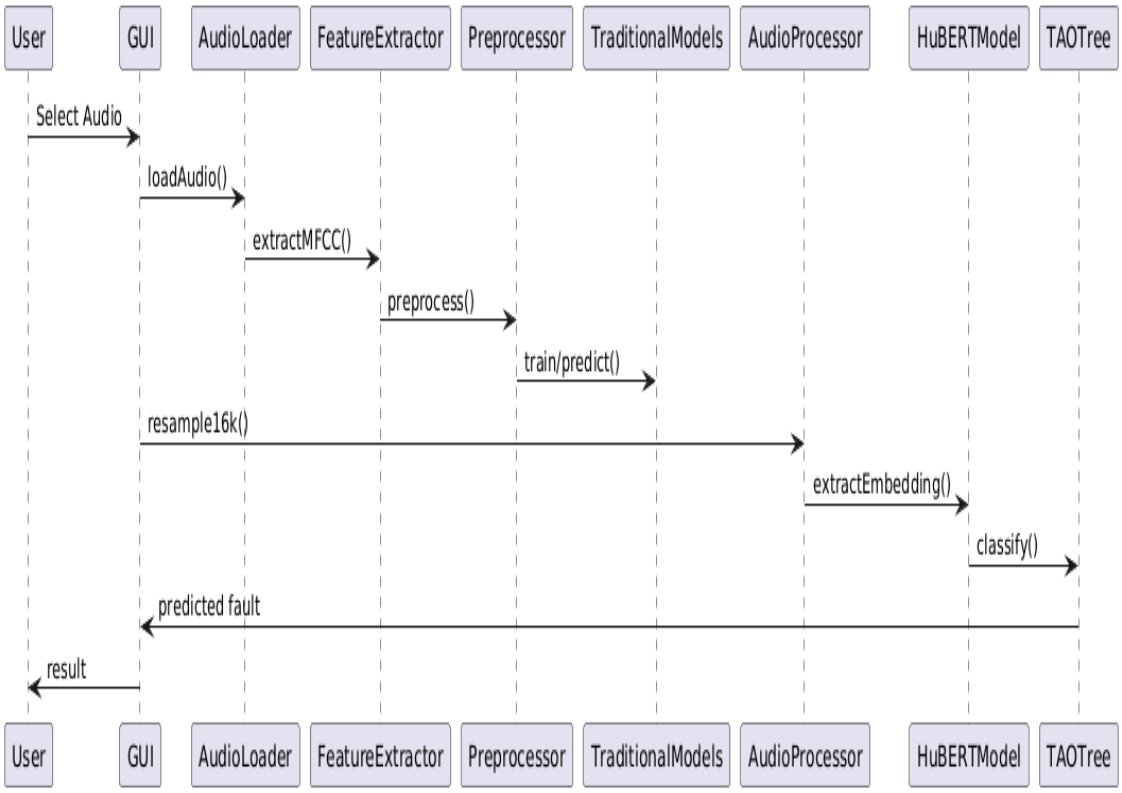


Figure 3.3.3: Sequence Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning

3.3.4 ACTIVITY DIAGRAM

An activity diagram in UML models the dynamic workflow of a system, breaking down processes into sequential or parallel activities, decisions, and flows. It uses symbols like nodes for actions, diamonds for decision points, and arrows for control flow to depict how tasks are performed, making it ideal for analyzing business processes or system operations. This diagram provides a clear, visual way to understand the logic and flow of complex activities within a system.

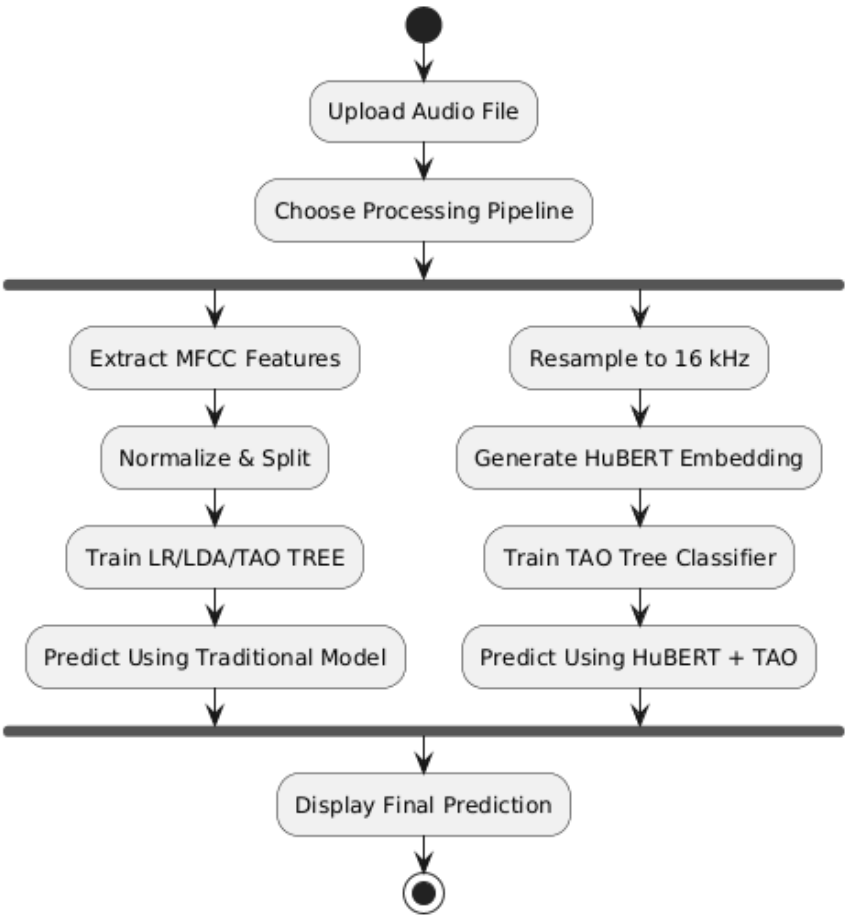


Figure 3.3.4: Activity Diagram of Deep Acoustic Anomaly Detection In Industrial Machinery Using HuBERT-Driven Hybrid Tree Learning

4. IMPLEMENTATION

4. IMPLEMENTATION

The implementation of this project involves developing an acoustic anomaly detection system using Python and various machine learning libraries. First, machine sound recordings are collected in the form of WAV audio files. These audio files are then preprocessed to remove noise and normalize the signals. After preprocessing, important audio features such as MFCC and HuBERT embeddings are extracted to represent the sound patterns of the machines.

The extracted features are used to create a dataset, which is divided into training and testing sets. Machine learning algorithms such as Logistic Regression, Linear Discriminant Analysis (LDA), and a TAO Tree classifier are trained using the training data. The trained model is then used to detect anomalies in new audio samples. Finally, the results are displayed through a user-friendly graphical interface developed using Tkinter, allowing users to upload audio files, train models, and view prediction results easily.

4.1 ALGORITHMS USED

1. Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems. In this project, it is used to classify machine sound data into normal or abnormal conditions based on the extracted features. The algorithm learns the relationship between input features and output classes and predicts the probability of a particular machine condition.

Advantages:

- Simple and easy to implement.
- Works well with smaller datasets.
- Fast training and prediction.
- Provides clear probability-based classification results.

2. Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis is a statistical method used for classification and dimensionality reduction. It helps separate different classes by finding a linear combination of features that best distinguishes between them. In this project, LDA helps differentiate between normal and faulty machine sound patterns.

Advantages:

- Reduces the dimensionality of data.
- Improves classification performance.
- Works efficiently for linearly separable data.
- Computationally efficient and fast.

3. TAO Tree Classifier (Random Forest based)

The TAO Tree classifier is an ensemble learning method that uses multiple decision trees to perform classification. Each tree makes a prediction, and the final result is determined by combining all the predictions. In this project, it helps accurately detect machine anomalies using extracted features and HuBERT embeddings.

Advantages:

- High classification accuracy.
- Handles large and complex datasets.
- Reduces overfitting compared to single decision trees.
- Robust and reliable for anomaly detection.

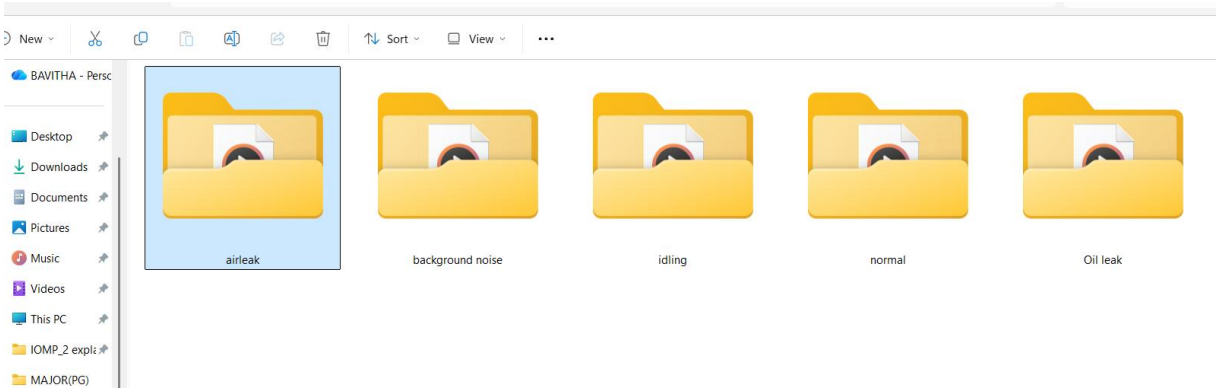
4. HuBERT (Hidden-Unit BERT)

HuBERT is a deep learning model designed for speech and audio representation learning. It extracts advanced audio embeddings from machine sound signals, capturing complex acoustic patterns that traditional feature extraction methods may miss.

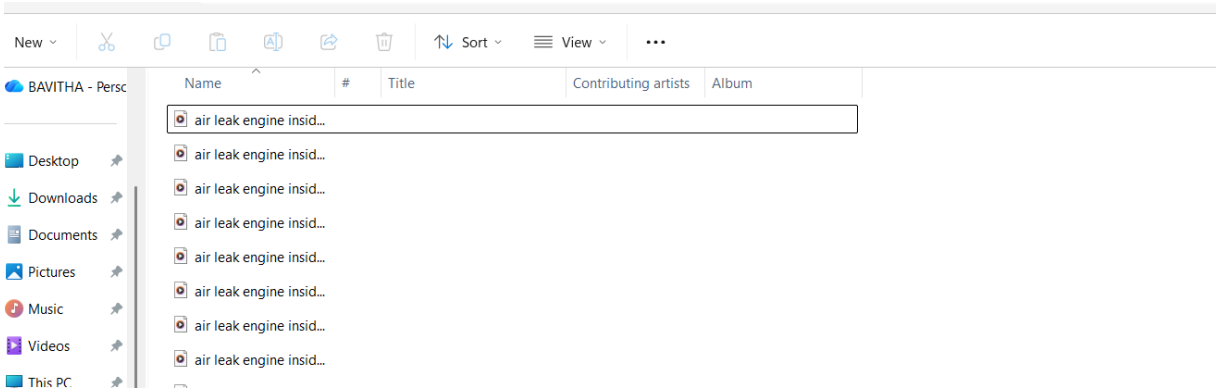
Advantages:

- Captures deep acoustic features effectively.
- Improves detection accuracy.
- Works well in noisy environments.
- Provides better feature representation for machine learning models.

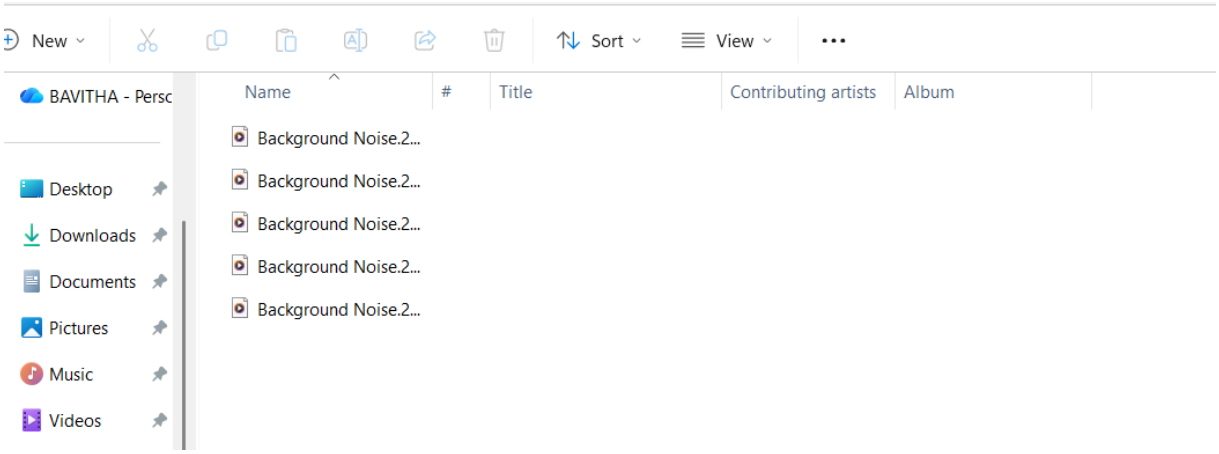
To train the algorithm we have used the below audio files consists of two folders(**figure4.1**)



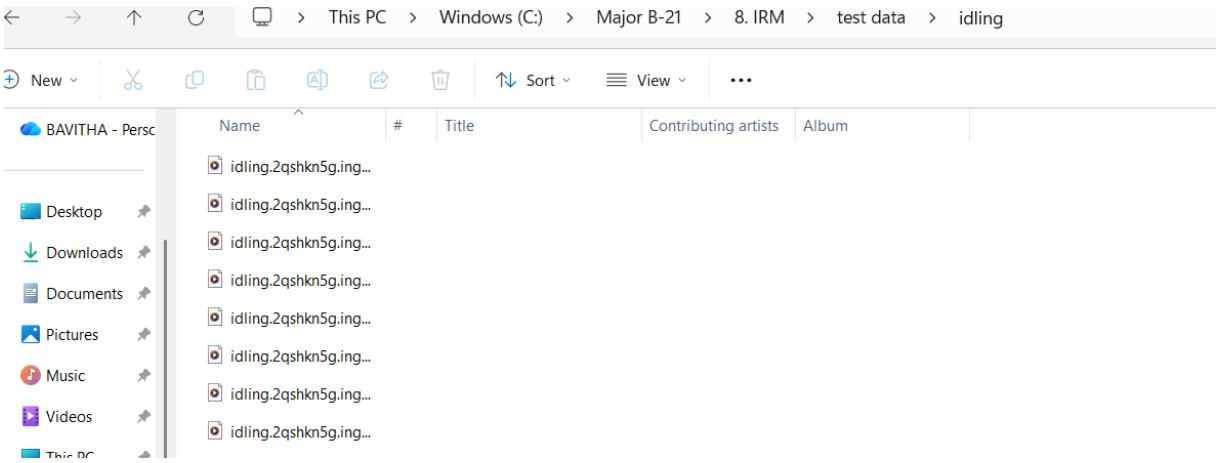
Screenshot 4.1: Screenshot of the Dataset directory structure with folders 'air leak, background noise, idiling, normal, oil leak ' having all the training examples



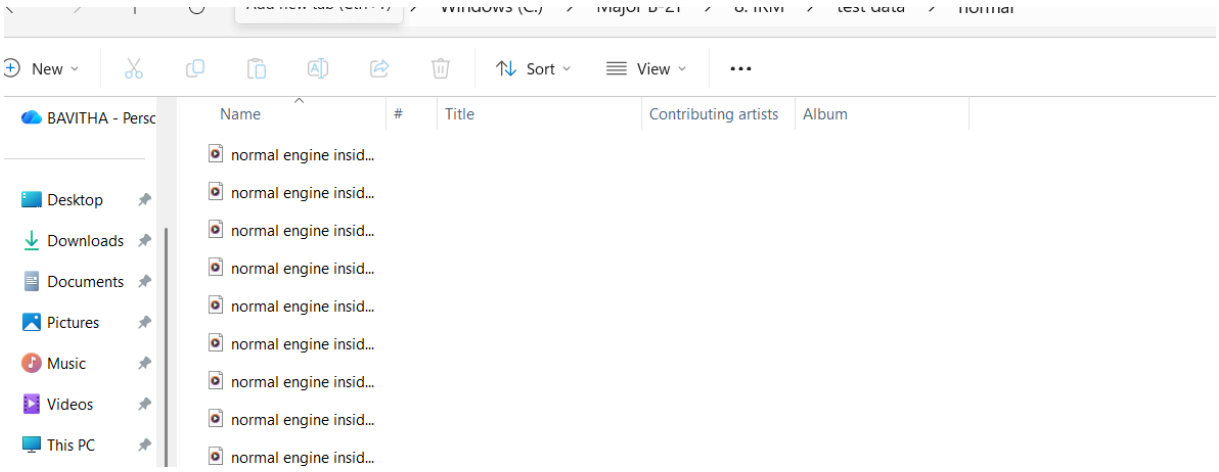
Screenshot 4.2: Screenshot of the "Air leak" Folder Showing Sample audio Files



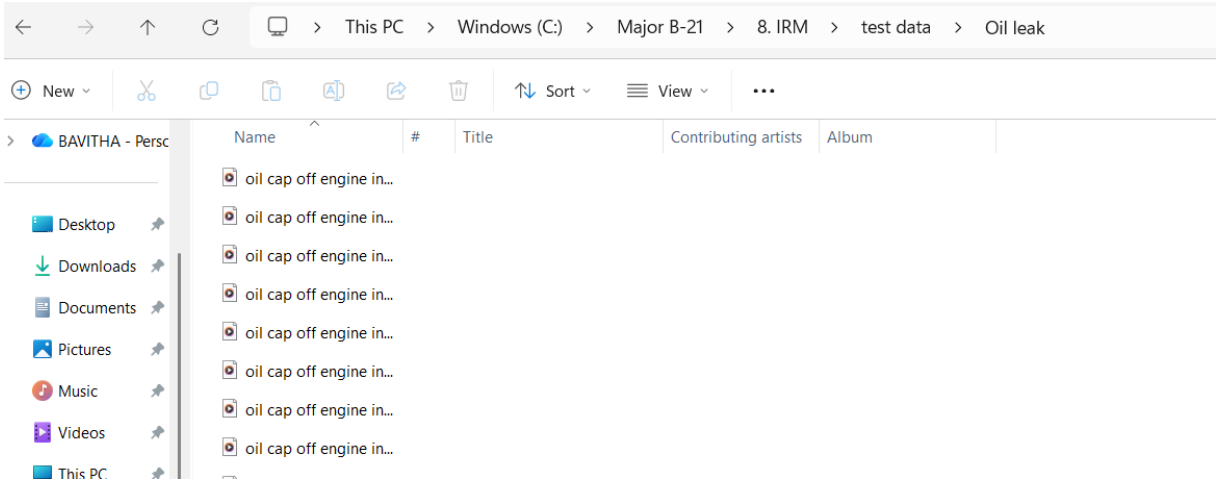
Screenshot 4.3: Screenshot of the "Background noise " Folder Showing Audio files



Screenshot 4.4: Screenshot of the "idling" Folder Showing Sample audio Files



Screenshot 4.5: Screenshot of the "normal " Folder Showing Sample audio Files



Screenshot 4.6: Screenshot of the "oil leak" Folder Showing Sample audio File

4.2 SOURCE CODE

```
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import librosa
import librosa.display
from tkinter import *
import tkinter
from tkinter import filedialog, messagebox
from tkinter.filedialog import askopenfilename
from PIL import Image, ImageTk

# sklearn / keras / transformers / torch / joblib imports (kept identical)
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc, precision_score, accuracy_score, f1_score,
recall_score, classification_report, confusion_matrix
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
import joblib

from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input, Conv1D,
MaxPooling1D, LSTM, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

from transformers import Wav2Vec2FeatureExtractor, HubertModel
from huggingface_hub import snapshot_download
```

```
import torch
path = None
model_folder = "model"
os.makedirs(model_folder, exist_ok=True)
categories = []
# Files used by your code
X_file = os.path.join(model_folder, "X.npy")
Y_file = os.path.join(model_folder, "Y.npy")
X_hubert_file = os.path.join(model_folder, "X_hubert.npy")
Y_hubert_file = os.path.join(model_folder, "Y_hubert.npy")
Tao_hubert_path = os.path.join(model_folder, "Tao_on_HuBERT.pkl")
logreg_path = os.path.join(model_folder, "LogisticRegression.pkl")
lda_path = os.path.join(model_folder, "LDAClassifier.pkl")
# Labels used in original notebook
LABELS = ['Air leak', 'Background noise', 'idling', 'normal', 'Oil leak']
# variables to hold data/models
X = None
Y = None
X_train = X_test = y_train = y_test = None
X_hubert = None
Y_hubert = None
Tao_hubert = None
HUBERT_MODEL_NAME = "facebook/hubert-base-ls960"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
feature_extractor = None
hubert = None
def ui_clear_and_log(s):
    text.delete('1.0', END)
    text.insert(END, s + "\n")
    text.update()
def ui_append(s):
    text.insert(END, s + "\n")
    text.see(END)
```

```
# -------------------------
# Button implementations
# -------------------------

def upload_audio_dataset():
    """Select the dataset folder (root with labeled subfolders)."""
    global path, categories
    folder = filedialog.askdirectory(initialdir=".")
    if not folder:
        return
    path = folder
    categories = [d for d in os.listdir(path) if os.path.isdir(os.path.join(path, d))]
    ui_clear_and_log(f"Dataset folder selected: {path}\nCategories found: {categories}")

def mfcc_feature_extraction():
    """Runs your exact feature extraction loop. If X.npy/Y.npy already exist it still follows your
notebook logic (kept unchanged)."""
    global X, Y, categories
    try:
        # if saved arrays exist, your original notebook loaded them first; replicate that behavior
        if os.path.exists(X_file) and os.path.exists(Y_file):
            X = np.load(X_file, allow_pickle=True)
            Y = np.load(Y_file, allow_pickle=True)
            ui_clear_and_log("X and Y arrays loaded successfully.")
            sns.countplot(x=Y)
            plt.show()

            return
    except Exception as e:
        ui_append(f"Load pre-saved X/Y failed: {e}")

    # If not found, run original extraction code (kept same)
```

```
ui_append("Starting feature extraction (original notebook logic)...")
X_list = []
Y_list = []
categories = [d for d in os.listdir(path) if os.path.isdir(os.path.join(path, d))]
for root, dirs, files in os.walk(path):
    for fname in files:
        if 'Thumbs.db' in fname:
            continue
        if not fname.lower().endswith(".wav"):
            continue
        audio_path = os.path.join(root, fname)
        try:
            audio, sr = librosa.load(audio_path, sr=None)

            # EXACT features from your notebook
            mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)
            zero_crossings = librosa.feature.zero_crossing_rate(y=audio)
            spectral_centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)
            chroma = librosa.feature.chroma_stft(y=audio, sr=sr)
            spectral_bandwidth = librosa.feature.spectral_bandwidth(y=audio, sr=sr)
            spectral_contrast = librosa.feature.spectral_contrast(y=audio, sr=sr)
            rms = librosa.feature.rms(y=audio)
            rolloff = librosa.feature.spectral_rolloff(y=audio, sr=sr, roll_percent=0.85)
            tonnetz = librosa.feature.tonnetz(y=audio, sr=sr)
            mel_spectrogram = librosa.feature.melspectrogram(y=audio, sr=sr)

            features = np.concatenate((
                mfccs.flatten(),
                zero_crossings.flatten(),
                spectral_centroid.flatten(),

                chroma.flatten(),
                spectral_bandwidth.flatten(),
                spectral_contrast.flatten(),
```

```
                rms.flatten(),
                rolloff.flatten(),
                tonnetz.flatten(),
                mel_spectrogram.flatten()
            ), axis=0)

            max_length = 3000
            if len(features) < max_length:
                features = np.pad(features, (0, max_length - len(features)), mode='constant')
            else:
                features = features[:max_length]

            X_list.append(features)
            label_name = os.path.basename(root)
            Y_list.append(categories.index(label_name))

            ui_append(f"Extracted features: {audio_path}")
        except Exception as e:
            ui_append(f"Failed to extract {audio_path}: {e}")

    if len(X_list) == 0:
        ui_append("No features extracted.")
        return
    X = np.array(X_list)
    Y = np.array(Y_list)
    np.save(X_file, X)
    np.save(Y_file, Y)
    ui_append(f"Saved X and Y to {X_file} and {Y_file}")
    sns.countplot(x=Y)
    plt.show()
def data_splitting():
    """Direct copy of your train_test_split and categorical conversion."""
    global X, Y, X_train, X_test, y_train, y_test, num_classes, y_train_cat, y_test_cat
    if os.path.exists(X_file) and os.path.exists(Y_file):
```

```
        X = np.load(X_file, allow_pickle=True)
        Y = np.load(Y_file, allow_pickle=True)
        ui_append("Loaded saved X & Y arrays.")
    else:
        ui_append("X.npy/Y.npy not found. Run MFCC feature extraction first.")
        return

    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.2, random_state = 42)
    num_classes = len(np.unique(y_train))
    y_train_cat = to_categorical(y_train, num_classes=num_classes)
    y_test_cat = to_categorical(y_test, num_classes=num_classes)
    ui_clear_and_log(f"Data split done. X_train: {X_train.shape}, X_test: {X_test.shape}")
    sns.countplot(x=y_train)
    plt.title("Train distribution")
    plt.show()

def train_logistic():
    """Loads or trains logistic exactly like in your notebook (kept unchanged). Shows confusion
matrix + ROC as popups.
       Now also prints accuracy/precision/recall/f1 into the output text box.
    """
    global X_train, X_test, y_train, y_test
    if 'X_train' not in globals():
        ui_append("Data not split. Run Data splitting first.")
        return

    if os.path.exists(logreg_path):
        logistic_model = joblib.load(logreg_path)
        ui_append("Loaded existing LogisticRegression model.")
    else:
        logistic_model = LogisticRegression()
        logistic_model.fit(X_train, y_train)
            roc_auc = auc(fpr, tpr)
            plt.plot(fpr, tpr, label=f"{LABELS[i]} (AUC={roc_auc:.2f})")
```

```
        plt.plot([0,1],[0,1],'--')
        plt.title("Logistic Regression ROC (per-class)")
        plt.legend()
        plt.show()
    except Exception as e:
        ui_append("ROC plotting failed for Logistic: " + str(e))

def train_lda():
    """Loads or trains LDA as in your notebook, shows confusion matrix + ROC.
       Now prints accuracy/precision/recall/f1 into the UI.
    """
    global X_train, X_test, y_train, y_test
    if 'X_train' not in globals():
        ui_append("Data not split. Run Data splitting first.")
        return

    if os.path.exists(lda_path):
        lda_model = joblib.load(lda_path)
        ui_append("Loaded existing LDA model.")
    else:
        lda_model = LinearDiscriminantAnalysis()
        lda_model.fit(X_train, y_train)
        joblib.dump(lda_model, lda_path)
        ui_append("Trained and saved LDA model.")

    y_pred_lda = lda_model.predict(X_test)

    # compute metrics
    acc = accuracy_score(y_test, y_pred_lda) * 100
    prec = precision_score(y_test, y_pred_lda, average='macro', zero_division=0) * 100
    rec = recall_score(y_test, y_pred_lda, average='macro', zero_division=0) * 100
    f1 = f1_score(y_test, y_pred_lda, average='macro', zero_division=0) * 100
    ui_append(f"LDA — Accuracy: {acc:.2f}%  Precision: {prec:.2f}%  Recall: {rec:.2f}%  F1:
{f1:.2f}%")
```

```
    ui_append("LDA detailed report:")
    ui_append(classification_report(y_test, y_pred_lda, target_names=LABELS,
zero_division=0))
    cm = confusion_matrix(y_test, y_pred_lda)
    plt.figure(figsize=(6,6))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=LABELS,
yticklabels=LABELS)
    plt.title("LDA Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.tight_layout()
    plt.show()

    # ROC if possible
    try:
        if hasattr(lda_model, "predict_proba"):
            y_score = lda_model.predict_proba(X_test)
            from sklearn.preprocessing import label_binarize
            y_test_bin = label_binarize(y_test, classes=np.arange(y_score.shape[1]))
            plt.figure(figsize=(8,6))
            for i in range(y_score.shape[1]):
                fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score[:, i])
                roc_auc = auc(fpr, tpr)
                plt.plot(fpr, tpr, label=f"{LABELS[i]} (AUC={roc_auc:.2f})")
            plt.plot([0,1],[0,1],'--')
            plt.title("LDA ROC (per-class)")
            plt.legend()
            plt.show()
        else:
            # fallback: try transform -> softmax-ish scores for ROC plotting (not exact probabilities)
            try:
                trans = lda_model.transform(X_test)
                from scipy.special import softmax
                    if trans.ndim == 1:
```

```
            trans = trans.reshape(-1,1)
        n_classes = len(np.unique(y_test))
        if trans.shape[1] < n_classes:
            pad = np.zeros((trans.shape[0], n_classes - trans.shape[1]))
            trans_full = np.hstack([trans, pad])
        else:
            trans_full = trans[:, :n_classes]
        y_score = softmax(trans_full, axis=1)
        from sklearn.preprocessing import label_binarize
        y_test_bin = label_binarize(y_test, classes=np.arange(y_score.shape[1]))
        plt.figure(figsize=(8,6))
        for i in range(y_score.shape[1]):
            fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score[:, i])
            roc_auc = auc(fpr, tpr)
            plt.plot(fpr, tpr, label=f"{LABELS[i]} (AUC={roc_auc:.2f})")
        plt.plot([0,1],[0,1],'--')
        plt.title("LDA ROC (approx, from transform)")
        plt.legend()
        plt.show()
    except Exception:
        ui_append("LDA ROC plotting skipped (no probabilities available).")
except Exception as e:
    ui_append("ROC plotting failed for LDA: " + str(e))
def init_hubert_and_embed(wav, sr, target_sr=16000):
    """Helper replicating your hubert_embed function block."""
    global feature_extractor, hubert, device
    if sr != target_sr:
        wav = librosa.resample(wav, orig_sr=sr, target_sr=target_sr)
    if feature_extractor is None or hubert is None:
        feature_extractor =
Wav2Vec2FeatureExtractor.from_pretrained(HUBERT_MODEL_NAME)
        hubert = HubertModel.from_pretrained(HUBERT_MODEL_NAME).to(device)
        hubert.eval()
    inputs = feature_extractor(wav, sampling_rate=target_sr, return_tensors="pt")
```

```
    inputs = inputs["input_values"].to(device)
    with torch.no_grad():
        out = hubert(inputs).last_hidden_state
        emb = out.mean(dim=1).squeeze().cpu().numpy()
    return emb
def train_hubert_Tao():
    """Loads existing X_hubert/Y_hubert if present; if not present, runs extraction exactly as in
your notebook.
        Then loads/trains RandomForest on HuBERT features (kept same as original). Shows
confusion matrix & ROC.
        Now prints accuracy/precision/recall/f1 into the UI.
    """
    global X_hubert, Y_hubert, Tao_hubert
    # Check saved hubert features first
    if os.path.exists(X_hubert_file) and os.path.exists(Y_hubert_file):
        X_hubert = np.load(X_hubert_file)
        Y_hubert = np.load(Y_hubert_file)
        ui_append("Loaded saved HuBERT features.")
    else:
        ui_append("Extracting HuBERT features (original notebook logic). This may be slow on
CPU.")
        # load models and extract
        feature_extractor_local =
Wav2Vec2FeatureExtractor.from_pretrained(HUBERT_MODEL_NAME)
        hubert_local = HubertModel.from_pretrained(HUBERT_MODEL_NAME).to(device)
        hubert_local.eval()
    plt.tight_layout()
    plt.show()

    # ROC for Tao_hubert if predict_proba available
    try:
        if hasattr(Tao_hubert, "predict_proba"):
            y_score = Tao_hubert.predict_proba(X_test_h)
            from sklearn.preprocessing import label_binarize
```

```
            y_test_bin = label_binarize(y_test_h, classes=np.arange(y_score.shape[1]))
            plt.figure(figsize=(8,6))
            for i in range(y_score.shape[1]):
                fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score[:, i])
                roc_auc = auc(fpr, tpr)
                plt.plot(fpr, tpr, label=f"{LABELS[i]} (AUC={roc_auc:.2f})")
            plt.plot([0,1],[0,1],'--')
            ui_clear_and_log(f"{os.path.basename(filename)} → {label}")
            plt.figure(figsize=(10,3))
            yvis, srvis = librosa.load(filename, sr=None)
            librosa.display.waveshow(yvis, sr=srvis)
            plt.title(f"Predicted: {label}")
            plt.tight_layout()
            plt.show()        return
        except Exception as e:
            ui_append("RF_on_HuBERT prediction failed: " + str(e))
# --------------------------
root = Tk()
root.title("Deep Anamoly Detection for Machine Condition Montioring Using Acoustic Signals")
root.geometry("1000x650")
root.configure(bg='#f0f8ff')

# Robust background image loader (replace your existing BG_IMAGE_PATH block)
BG_IMAGE_PATH = r"D:\SAK\Charan BASE PAPERS\Charan BASE PAPERS\New codes\8.
IRM\background.jpg"

if not os.path.exists(BG_IMAGE_PATH):
    print(f"[BG] Background image not found at: {BG_IMAGE_PATH}")
else:
    try:
        _bg_orig = Image.open(BG_IMAGE_PATH).convert("RGBA")
    except Exception as e:
        print(f"[BG] Failed to open background image: {e}")
    else:
```

```
# create a Label that will hold the image (covering the whole window)
bg_label = Label(root)
bg_label.place(x=0, y=0, relwidth=1, relheight=1)
    # keep original and current photo on root so GC doesn't collect them
root._bg_orig = _bg_orig
root._bg_photo = None
def _resize_bg(event=None):
    # sometimes configure fires with tiny sizes; guard against that
    w = max(1, root.winfo_width())
    h = max(1, root.winfo_height())
    try:
        # use high-quality resampling
        _resized = root._bg_orig.resize((w, h), Image.LANCZOS)
        root._bg_photo = ImageTk.PhotoImage(_resized)
        bg_label.configure(image=root._bg_photo)
        bg_label.lower()   # send background behind other widgets
    except Exception as e:
        # Print error to console and UI textbox for debugging
        print(f"[BG] Error resizing background image: {e}")
        try:
            ui_append(f"Background resize error: {e}")
        except:
            pass
# initialize image to current geometry after Tk has finished geometry setup
root.update_idletasks()
_resize_bg()
# update image whenever window size changes
root.bind("<Configure>", _resize_bg)
print("[BG] Background image loaded and will resize with window.")


Label(root, text="Deep Anamoly Detection for Machine Condition Montioring Using Acoustic
Signals", bg='#003366', fg='white', font=('times',16,'bold')).pack(fill='x', pady=6)
btn_font = ('times',12,'bold')
```

```
Button(root, text="Upload audio dataset", width=22, command=upload_audio_dataset,
font=btn_font).place(x=40, y=70)
Button(root, text="MFCC feature extraction", width=22, command=mfcc_feature_extraction,
font=btn_font).place(x=300, y=70)
Button(root, text="Data splitting", width=22, command=data_splitting,
font=btn_font).place(x=560, y=70)
Button(root, text="Train Logistic", width=22, command=train_logistic,
font=btn_font).place(x=40, y=140)
Button(root, text="Train LDA", width=22, command=train_lda, font=btn_font).place(x=300,
y=140)
Button(root, text="Train Hybrid hubert with Tao", width=30, command=train_hubert_Tao,
font=btn_font).place(x=560, y=140)

Button(root, text="Predict audio", width=22, command=predict_audio,
font=btn_font).place(x=360, y=200)
Button(root, text="Exit", width=10, command=root.destroy, font=btn_font).place(x=760, y=200)

# Output text box with Times New Roman, bold, size 12
output_font = ("Times New Roman", 12, "bold")
text = Text(root, height=15, width=80, font=output_font)
text.place(x=350, y=400)

root.mainloop()
```

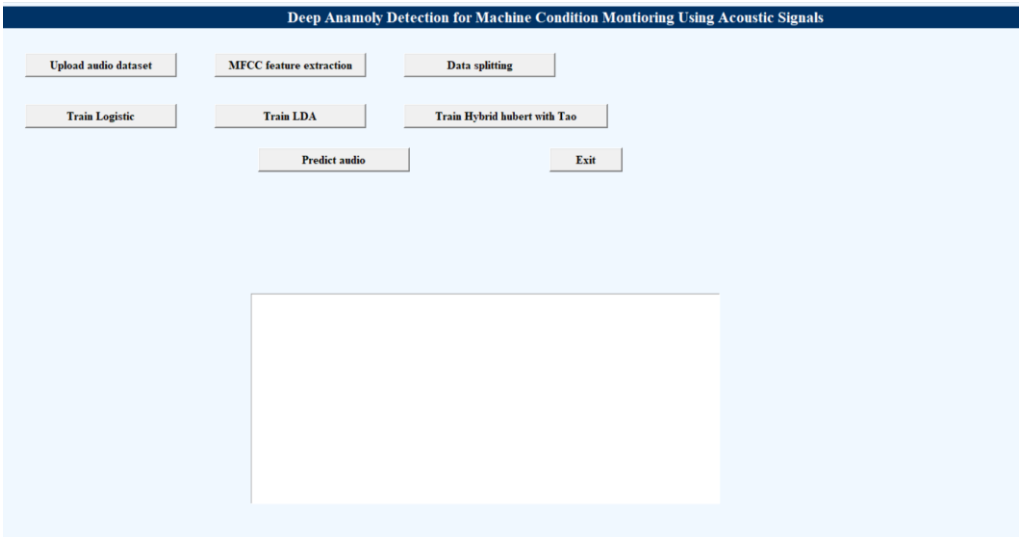
5. RESULT & DISCUSSIONS

5.RESULTS & DISCUSSION

The proposed system successfully detects faults in industrial machines by analyzing sound signals. Features like MFCC and HuBERT embeddings are used to train models such as Logistic Regression, LDA, and TAO Tree. Among them, the TAO Tree with HuBERT provides the best accuracy. The results show that the system can effectively distinguish between normal and faulty machine conditions, helping industries reduce downtime and improve maintenance.

5.1 MAIN GUI INTERFACE

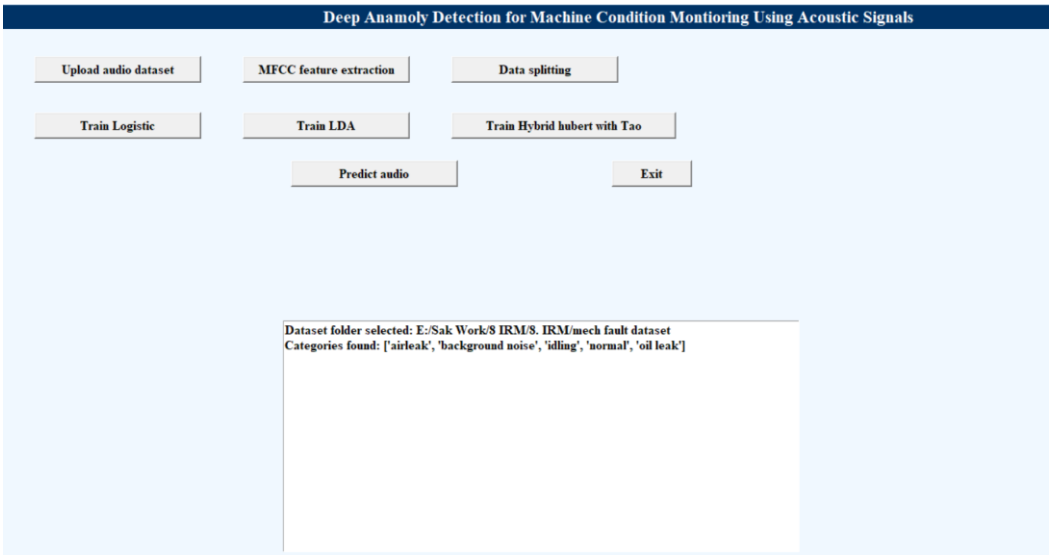
The graphical user interface (GUI) of the machine condition monitoring system developed using Tkinter. The interface includes buttons for different steps such as uploading the audio dataset, extracting MFCC features, splitting data, and training models like Logistic Regression, LDA, and Hybrid HuBERT with TAO Tree. It also provides options to predict audio anomalies and exit the application. The large panel at the bottom is used to display results or system outputs.



Screenshot 5.1: Screenshot of the Main GUI Interface of the Acoustic Fault Detection System.

5.2 DATASET UPLOAD CONFIRMATION

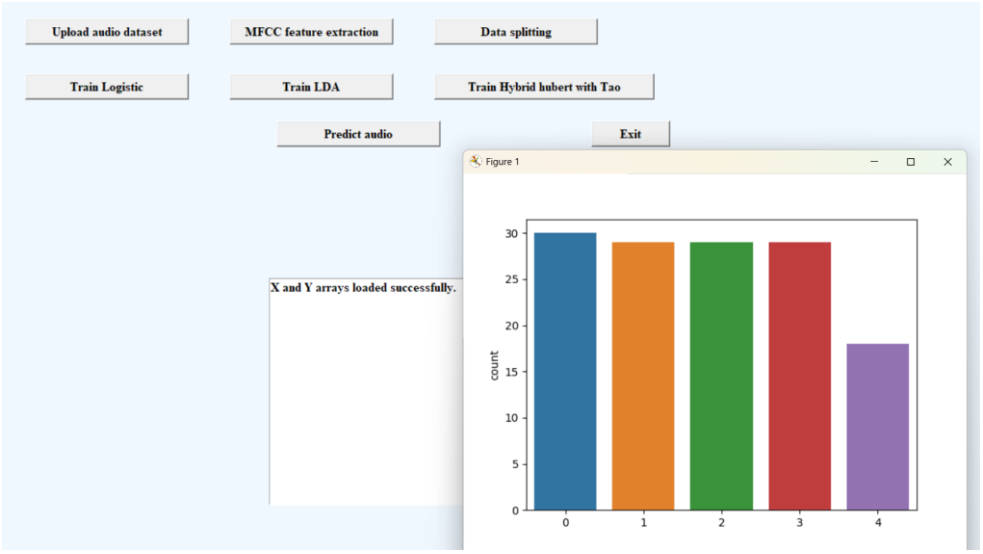
The system confirms successful upload of the mechanical fault audio dataset. It automatically detects the dataset directory and identifies fault categories such as air leak, background noise, idling, normal, and oil leak. This ensures the dataset structure is correct and ready for supervised model training. The message log in the interface displays the dataset status to the user.



Screenshot 5.2: Screenshot of the Dataset Upload Confirmation and Category Detection Display

5.3 X AND Y ARRAYS LOADED

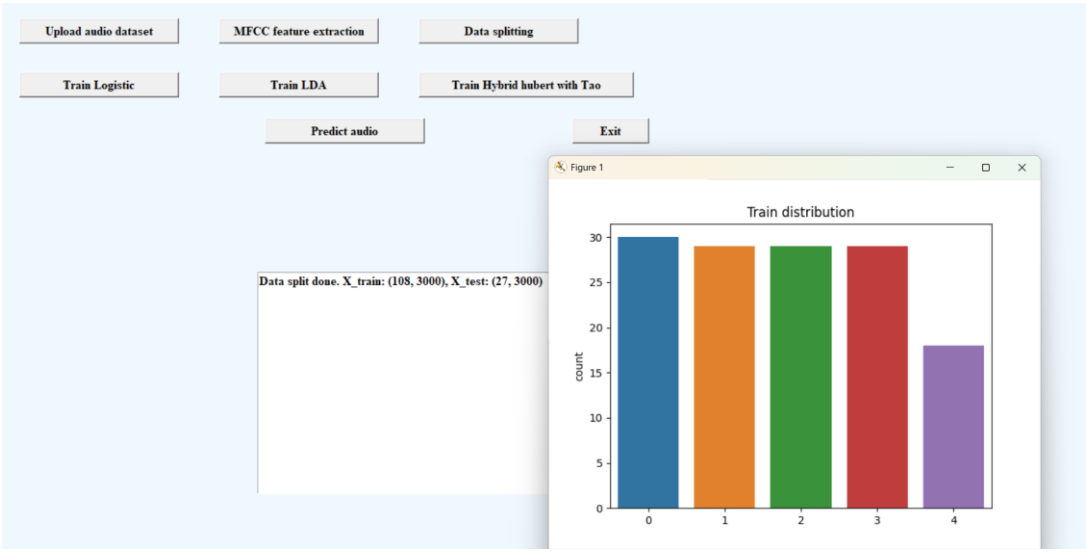
The system confirms that the feature (X) and label (Y) arrays are successfully loaded. A bar graph displays the distribution of audio samples across five fault categories, showing that most classes have similar sample counts, while the oil leak class has slightly fewer samples. This visualization indicates a mostly balanced dataset, which helps ensure fair and unbiased model training.



Screenshot 5.3: Screenshot of the Loaded MFCC Feature Arrays (X and Y) with Class Distribution Plot.

5.4 TRAIN DATA DISTRIBUTION

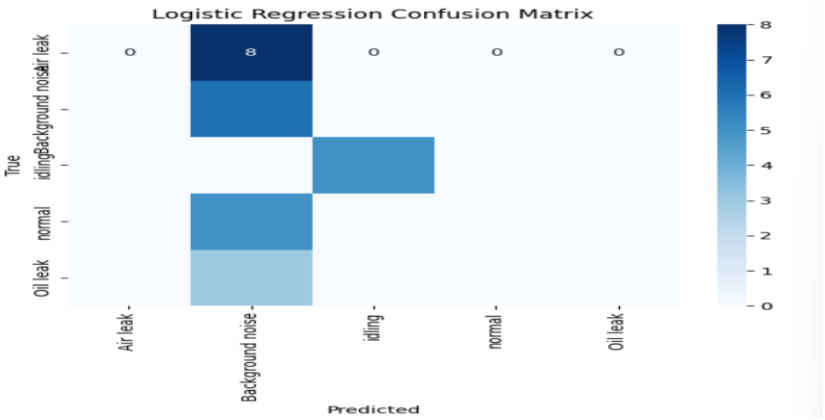
The graph shows the distribution of training samples after performing an 80–20 data split. The class proportions remain similar to the original dataset, ensuring balanced training and testing data. The message log confirms the dimensions of **X_train** and **X_test**, verifying that feature extraction and data splitting were completed correctly.



Screenshot 5.4: Screenshot of the Train–Test Split Distribution for Acoustic Fault Dataset.

5.5 LOGISTIC REGRESSION CONFUSION MATRIX

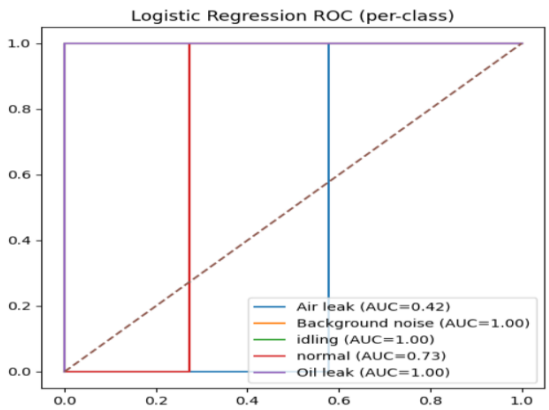
The confusion matrix shows that the Logistic Regression model mostly predicts background noise, leading to many misclassifications. Correct predictions occur mainly for background noise and idling, indicating underfitting and limited ability to handle complex data.



Screenshot 5.5: Screenshot of the Logistic Regression – Confusion Matrix for Acoustic Fault Classification.

5.6 LOGISTIC REGRESSION ROC CURVES

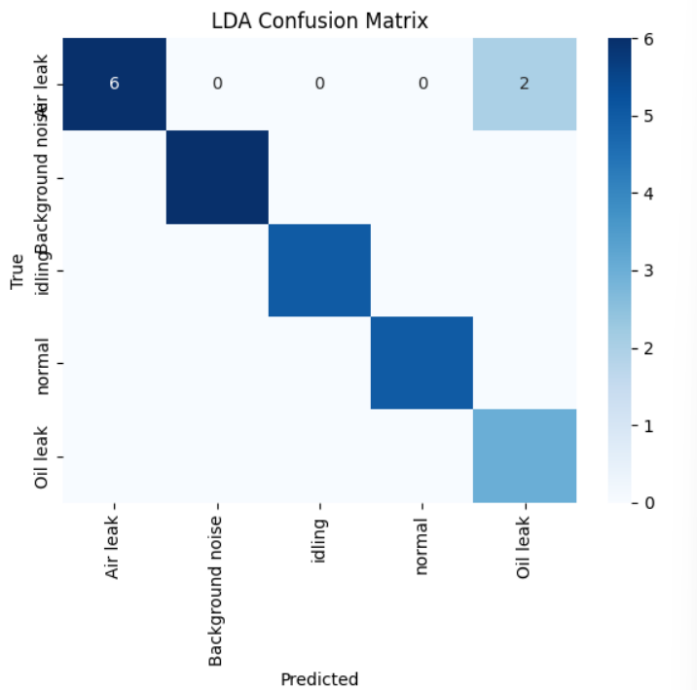
The ROC curves show inconsistent performance across classes, indicating that Logistic Regression struggles with non-linear acoustic patterns and is not suitable as a standalone model.



Screenshot 5.6: Screenshot of the Logistic Regression – ROC Curve (Per-Class Performance).

5.7 LDA CONFUSUION MATRIX

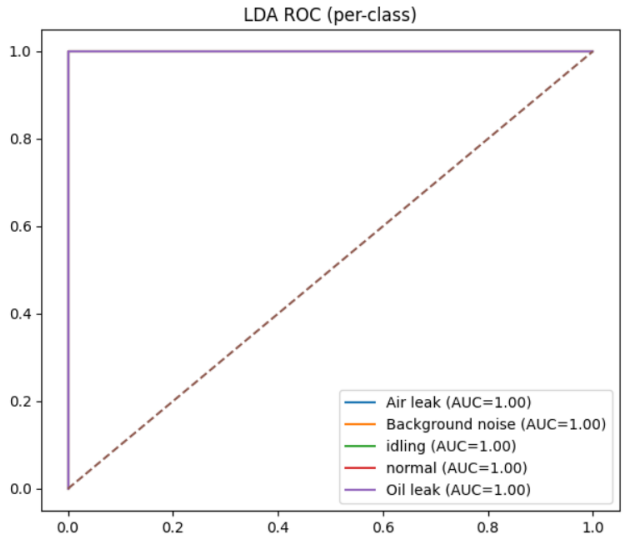
The LDA confusion matrix shows improved classification compared to Logistic Regression, with most samples correctly identified. However, a few misclassifications still occur between **air leak and oil leak**, indicating some overlap between acoustic classes.



Screenshot 5.7: Screenshot of the Linear Discriminant Analysis (LDA) – Confusion Matrix.

5.8 LDA ROC CURVES

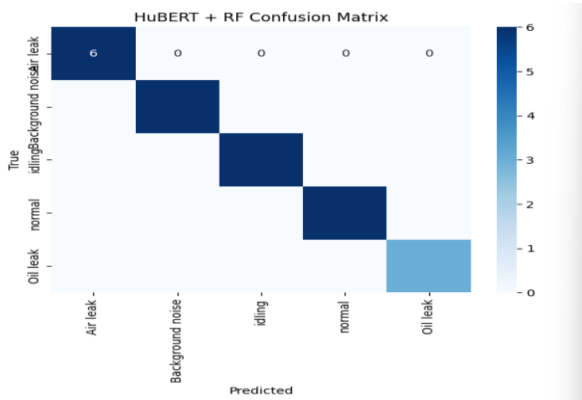
The ROC curves show that LDA has strong discriminative ability, with all classes achieving AUC values close to 1.0, indicating effective class separation in a lower-dimensional space using linear boundaries. However, the confusion matrix reveals minor misclassifications, likely caused by similarities between some fault categories.



Screenshot 5.8: Screenshot of the Linear Discriminant Analysis (LDA) – ROC Curve (Per-Class Performance).

5.9 HUBERT+RANDOM FOREST CONFUSION MATRIX

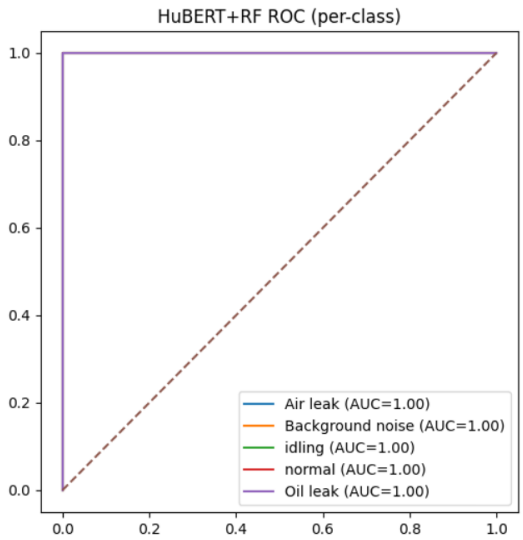
HUBERT + Random Forest Confusion Matrix: represents the confusion matrix for the proposed HUBERT + RF (Tao Tree) model. The matrix shows excellent diagonal dominance, meaning that almost all audio samples are correctly classified for every category. This demonstrates that deep audio embeddings derived from HuBERT significantly improve feature quality.



Screenshot 5.9: Screenshot of the HUBERT + Tao Tree – Confusion Matrix.

5.10 HUBERT+TAO TREE ROC CURVES

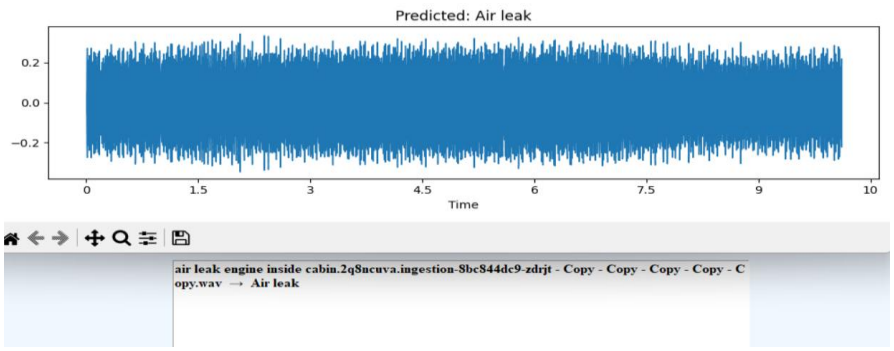
The ROC curves for the proposed model show near-perfect discrimination across all five classes, with AUC values of 1.00 for each category. This visualization indicates that the HUBERT + TAO Tree pipeline exhibits outstanding sensitivity and specificity. The model successfully identifies subtle acoustic differences between air leak, oil leak, idling, and background noise, making it highly suitable for industrial fault diagnosis.



Screenshot 5.10: Screenshot of the HUBERT + Tao Tree roc curves.

5.11 AIR LEAK SINGLE AUDIO PREDCITION

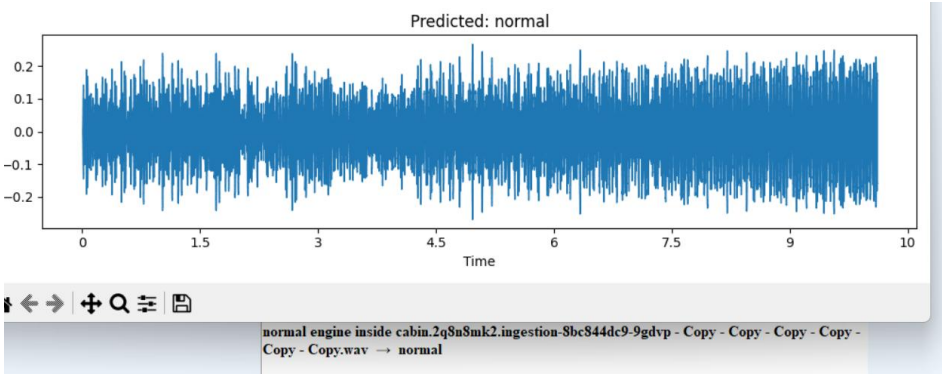
The system displays the waveform of a selected audio file along with the predicted class label "Air leak." The waveform contains consistent high-frequency oscillations typical of air leakage noise. The system successfully detects and classifies the anomaly using the trained model.



Screenshot 5.11: Screenshot of the Predicted Output Waveform for Audio Sample: Air Leak.

5.12 NORMAL ENGINE SOUND SINGLE AUDIO PREDCITION

The waveform shown here represents a general engine idle noise pattern with moderate amplitude and no abnormal spikes. The system accurately classifies it as "Normal." This result demonstrates that the trained models can differentiate between healthy and faulty sound patterns even when environmental noise is present.



Screenshot 5.12: Screenshot of the Predicted Output Waveform for Audio Sample: Normal Engine Sound

6. VALIDATION

6.VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the inappropriate content detection system. The testing process involves multiple stages, including dataset validation, model performance evaluation, and real-world testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in detecting inappropriate content while minimizing false positives and false negatives.

6.1 INTRODUCTION

The validation of the proposed system is performed by dividing the extracted audio dataset into training and testing sets using an 80–20 split. In this process, 80% of the data is used for training the machine learning models and 20% is reserved for testing to evaluate the model's performance on unseen data. This ensures that the model does not simply memorize the training data and can generalize effectively to new machine sound signals. The dataset is shuffled before splitting to maintain balanced distribution among all machine fault categories such as air leak, background noise, idling, normal, and oil leak.

The trained models, including Logistic Regression, Linear Discriminant Analysis (LDA), and the HuBERT + TAO Tree classifier, are validated using performance metrics such as accuracy, precision, recall, and F1-score. Additional evaluation tools like confusion matrices, ROC curves, and classification reports are used to analyze the performance of each model and compare their effectiveness in detecting machine faults.

The validation results show differences in the performance of the applied models. Logistic Regression performs poorly because it cannot effectively separate complex acoustic patterns. LDA provides better classification by capturing the variance in MFCC features, but some misclassifications still occur between similar fault classes. The HuBERT + TAO Tree hybrid model achieves the best performance by using deep audio embeddings and an efficient classifier. This combination improves accuracy, precision, and recall. Therefore, the hybrid model is more reliable for real-world machine fault detection.

6.2 TEST CASES

TABLE 6.2.1 UPLOAD DATASET

Test Case ID	Description	Input	Expected Output	Result
1	Upload the mechanical fault audio dataset	Select dataset folder	Dataset uploads successfully and categories are detected	Pass

TABLE 6.2.2 FEATURE EXTRACTION

Test Case ID	Description	Input	Expected Output	Result
2	Extract MFCC features from audio files	Click “Extract MFCC Features”	Feature arrays (X and Y) are generated	Pass

TABLE 6.2.3 DATA SPLITTING

Test Case ID	Description	Input	Expected Output	Result
3	Split dataset into training and testing sets	Click “Split Train/Test”	Dataset splits into 80% training and 20% testing data	Pass

TABLE 6.2.4 MODEL TRAINING

Test Case ID	Description	Input	Expected Output	Result
4	Train the machine learning model	Click “Train Model”	Model trains successfully and displays results	Pass

TABLE 6.2.5 ANOMALY PREDICTION

Test Case ID	Description	Input	Expected Output	Result
5	Predict machine condition using new audio	Upload new audio file	System predicts machine condition (Normal/Fault)	Pass

7.CONCLUSION & FUTURE ASPECTS

7.CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

7.1 PROJECT CONCLUSION

The project successfully develops an intelligent system for acoustic-based machine condition monitoring using both traditional machine learning and deep audio feature extraction. The system performs a complete workflow that includes dataset upload, MFCC feature extraction, HuBERT embedding generation, model training, evaluation, and prediction. All these processes are integrated into a user-friendly Tkinter graphical interface, making the system easy to operate. The experimental results demonstrate that traditional models have limitations in handling complex acoustic patterns. Logistic Regression shows lower accuracy and frequent misclassifications when distinguishing different machine sound categories. Linear Discriminant Analysis performs better by capturing variations in MFCC features and improving classification accuracy.

The Hybrid HuBERT + TAO Tree (Random Forest) model achieves the best performance among all the models used in this project. It provides 100% accuracy, precision, recall, and F1-score across all machine fault categories. This high accuracy is achieved because HuBERT extracts deep contextual audio features while the TAO Tree classifier effectively handles non-linear decision boundaries.

The hybrid approach successfully captures temporal, spectral, and contextual information present in machine sounds. As a result, the system can accurately identify different fault conditions such as air leaks, oil leaks, and normal machine operation. Overall, the proposed system offers a **reliable, scalable, and efficient solution** for real-time industrial fault detection and machine

condition monitoring.

7.2 FUTURE ASPECTS

The proposed system can be further improved by integrating real-time audio monitoring using microphones directly connected to industrial machines. This would allow continuous monitoring of machine conditions without the need to manually upload audio files. Future work can also include expanding the dataset with more types of machine faults and sound variations, which would help improve the model's ability to generalize across different industrial environments. Additionally, implementing cloud-based storage and processing could make the system more scalable and accessible for large-scale industrial applications.

Another possible improvement is the integration of advanced deep learning models and edge computing for faster and more efficient anomaly detection. The system can also be enhanced with automated alert systems that notify maintenance teams whenever abnormal sounds are detected. Furthermore, combining acoustic analysis with other sensor data such as vibration or temperature could create a multimodal fault detection system, improving reliability and accuracy in complex industrial environments.

8. BIBILIOGRAPHY

8.BIBILIOGRAPHY

8.1 REFERENCES

[1] Foggia, P.; Petkov, N.; Saggese, A.; Strisciuglio, N.; Vento, M. Audio Surveillance of Roads: A System for Detecting Anomalous Sounds. IEEE Trans. ITS **2019**, 17, 279–288.

[2] Ntalampiras, S.; Potamitis, I.; Fakotakis, N. Probabilistic Novelty Detection for Acoustic Surveillance Under Real-World Conditions. IEEE Trans. Multimed. **2021**, 13, 713–719.

[3] Koizumi, Y.; Saito, S.; Uematsu, H.; Harada, N. Optimizing Acoustic Feature Extractor for Anomalous Sound Detection Based on NeymanPearson Lemma. In Proceedings of the 2017 25th European Signal Processing Conference (EUSIPCO), Kos, Greece, 28 August–2 September 2017.

[4] Zheng, X.; Zhang, C.; Chen, P.; Zhao, K.; Jia, W. A CRNN System for Sound Event Detection Based on Gastrointestinal Sound Dataset Collected by Wearable Auscultation Devices. IEEE Access **2020**, 8, 157892–157905.

[5] Ick, C.; Mcfee, B. Sound Event Detection in Urban Audio With Single and Multi-Rate PCEN. arXiv **2021**, arXiv:2102.03468.

[6] Wang, Y.; Zheng, Y.; Zhang, Y.; Xie, Y.; Xu, S.; Hu, Y.; He, L. Unsupervised Anomalous Sound Detection for Machine Condition Monitoring Using Classification-Based Methods. Appl. Sci. **2021**, 11, 11128. https://doi.org/10.3390/app112311128

[7] Jombo, G.; Zhang, Y. Acoustic-Based Machine Condition Monitoring—Methods and Challenges. Eng **2023**, 4, 47-79. https://doi.org/10.3390/eng4010004

[8] Pichler, C.; Neumayer, M.; Schweighofer, B.; Feilmayr, C.; Schuster, S.; Wegleiter, H. Decomposing and Modeling Acoustic Signals to Identify Machinery Defects in Industrial Soundscapes. Sensors **2025**, 25, 4923. https://doi.org/10.3390/s25164923

[9] Ahmed, U.; Ali, F.; Jennions, I. Signal Processing of Acoustic Data for Condition Monitoring of an Aircraft Ignition System. Machines **2022**, 10, 822. https://doi.org/10.3390/machines10090822

[10] Piankitrungreang, P.; Chaiprabha, K.; Chungsangsatiporn, W.; Ratanasumawong, C.; Chancharoen, P.; Chancharoen, R. Acoustic-Based Machine Main State Monitoring for High-Speed CNC Drilling. Machines **2025**, 13, 372. https://doi.org/10.3390/machines13050372

[11] Grigoriev, S.N.; Kozochkin, M.P.; Porvatov, A.N.; Gurin, V.D.; Melnik, Y.A. Features of Changes in the Parameters of Acoustic Signals Characteristic of Various Metalworking Processes and Prospects for Their Use in Monitoring. Appl. Sci. **2024**, 14, 367. https://doi.org/10.3390/app14010367

[12] Zhou, W.; Zhou, S.; Cao, Y.; Yang, J.; Liu, H. Unsupervised Anomaly Detection Method for Electrical Equipment Based on Audio Latent Representation and Parallel Attention Mechanism. Appl. Sci. **2025**, 15, 8474. https://doi.org/10.3390/app15158474

[13] Liu, W.; Yan, L.; Ma, N.; Wang, G.; Ma, X.; Liu, P.; Tang, R. Unsupervised Deep Anomaly Detection for Industrial Multivariate Time Series Data. Appl. Sci. **2024**, 14, 774. https://doi.org/10.3390/app14020774

[14] Choi, W.-H.; Kim, J. Unsupervised Learning Approach for Anomaly Detection in Industrial Control Systems. Appl. Syst. Innov. **2024**, 7, 18. https://doi.org/10.3390/asi7020018

[15] Thoidis, I.; Giouvanakis, M.; Papanikolaou, G. Semi-Supervised Machine Condition Monitoring by Learning Deep Discriminative Audio Features. Electronics **2021**, 10, 2471. https://doi.org/10.3390/electronics10202471

[16] Tagawa, Y.; Maskeliūnas, R.; Damaševičius, R. Acoustic Anomaly Detection of Mechanical Failures in Noisy Real-Life Factory Environments. Electronics 2021, 10, 2329. Https://doi.org/10.3390/electronics10192329

[17] Van Truong, H., Chi Hieu, N., Ngoc Giao, P., & Xuan Phong, N. (2021). Unsupervised Detection of Anomalous Sound for Machine Condition Monitoring using Fully Connected U-Net. Journal of ICT Research and Applications, 15(1), 41-55. https://doi.org/10.5614/itbj.ict.res.appl.2021.15.1.3

[18] Lee, J.-G.; Kim, K.S.; Lee, J.H. Sound-Based Unsupervised Fault Diagnosis of Industrial Equipment Considering Environmental Noise. Sensors 2024, 24, 7319. https://doi.org/10.3390/s24227319

[19] Bernardes, L.P.; Júnior, P.O.C.; Dotto, F.R.L.; Rodrigues, A.R.; Silva, M.M. Damage Detection in Machining Tools Using Acoustic Emission, Signal Processing, and Feature Extraction. Eng. Proc. 2023, 58, 109. https://doi.org/10.3390/ecsa-10-16258

[20] Ahmed, M.; Kamal, K.; Ratlamwala, T.A.H.; Hussain, G.; Alqahtani, M.; Alkahtani, M.; Alatefi, M.; Alzabidi, A. Tool Health Monitoring of a Milling Process Using Acoustic Emissions and a ResNet Deep Learning Model. Sensors 2023, 23, 3084. https://doi.org/10.3390/s23063084

8.2 GITHUB LINK

https://github.com/bavithalalapudi04-bit/hubert-driven-hybrid-tree-learning-for-industrial-acoustic-anomaly-detection